# Python & Algorithms for Coding Interviews

Section 1 — Python for Algorithmic Thinking

## 1.1 Python Execution Model

### 1.1.1 How Python Runs Your Code

Most people think Python just 'reads and runs' code line by line. That mental model is incomplete — and it causes confusion when debugging recursive functions, understanding scope, or analysing performance.

**Here is what actually happens when you run a Python file.**

**Step 1 — Source Code:** You write solution.py. This is plain text.

**Step 2 — Compilation to Bytecode:** Python compiles source code into bytecode — a lower-level, platform-independent instruction set. Stored in .pyc files inside __pycache__.

**Step 3 — Execution by the PVM:** The Python Virtual Machine (PVM) reads the bytecode and executes it instruction by instruction.

```
Your Code (.py)
      ↓
  Compiler
      ↓
 Bytecode (.pyc)
      ↓
Python Virtual Machine (PVM)
      ↓
   Output
```

| | |
|---|---|
| **Interview** | When asked "Is Python interpreted or compiled?" the correct answer is both. It compiles to bytecode first, then the PVM interprets that bytecode. This detail impresses interviewers. |

### 1.1.2 The Interpreter and Bytecode

The default Python implementation is CPython — written in C. When you type python in your terminal, you are running CPython. You can inspect the bytecode Python generates using the dis module.

```
import dis

def add(a, b):
    return a + b

dis.dis(add)

# Output:
# LOAD_FAST   a
# LOAD_FAST   b
# BINARY_ADD
# RETURN_VALUE
```

Each line is a bytecode instruction. The PVM executes these one by one. Python does not compile to machine code directly — every bytecode instruction goes through the PVM. This is one reason Python is slower than compiled languages, and why algorithm efficiency matters more in Python.

| | |
|---|---|
| **Terms** | CPython · bytecode · PVM · interpreted language |

### 1.1.3 The Call Stack

This is one of the most important mental models for understanding recursion, function calls, and stack overflow errors. Every time you call a function, Python creates a stack frame and pushes it onto the call stack. When the function returns, the frame is popped off.

```
def c():
    return 1

def b():
    return c()

def a():
    return b()

a()

# Call stack during execution:
```

```
# |   c()    |  ← top (currently executing)
# |   b()    |
# |   a()    |
# | main()   |  ← bottom
```

**Each frame stores:** local variables · return address · function arguments

**Recursion limit:** Python's default is 1000 frames. Hitting it means you need an iterative solution with an explicit stack — always safer in interviews.

```
import sys
sys.setrecursionlimit(10000)  # increase if needed
```

### 1.1.4 Stack vs Heap Memory

| Memory Region | Stores | Size | Speed |
|---|---|---|---|
| Stack | Function frames, local variable references | Small, fixed | Very fast |
| Heap | Actual objects (lists, dicts, instances) | Large, dynamic | Slower |

**Key insight:** Variables do not store objects directly. They store references (pointers) to objects living on the heap. This is why assignment does not copy objects — it copies references.

```
a = [1, 2, 3]
b = a          # b points to the SAME list on the heap
b.append(4)
print(a)       # [1, 2, 3, 4]  — a is affected too!
```

## 1.2 Python Memory Model

### 1.2.1 Mutable vs Immutable Objects

This is one of the most tested Python concepts in interviews — and one of the most misunderstood.

| Category | Types | Behaviour on Change |
|---|---|---|
| Immutable | int, float, str, tuple, bool, frozenset | Creates a new object |

| | | |
|---|---|---|
| Mutable | list, dict, set, class instances | Modifies in place |

```
# Immutable — string
s = "hello"
s += " world"   # creates a NEW string object; original unchanged

# Mutable — list
nums = [1, 2, 3]
nums.append(4)  # modifies the SAME list object in memory
```

| | |
|---|---|
| ⬚ **Trap** | String concatenation inside a loop is O(n²). Always use "".join() — it is O(n). |

```
# BAD — O(n²)
result = ""
for char in s:
    result += char   # new string every iteration

# GOOD — O(n)
parts = []
for char in s:
    parts.append(char)
result = "".join(parts)
```

### 1.2.2 Object Identity vs Equality (is vs ==)

| Operator | Checks | Example |
|---|---|---|
| == | Value equality | [1,2] == [1,2] → True |
| is | Same object in memory | [1,2] is [1,2] → False |

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a == b)   # True  — same values
print(a is b)   # False — different objects in memory
print(a is c)   # True  — c points to the same object as a

# CPython integer caching (-5 to 256)
x = 100; y = 100
print(x is y)   # True  — cached object

x = 1000; y = 1000
print(x is y)   # False — new objects
```

| | |
|---|---|
| ⬚ **Rule** | Never use 'is' to compare values. Use ==. Reserve 'is' only for: if x is None. |

### 1.2.3 Reference Counting

Every Python object has a counter tracking how many references point to it. When the count reaches zero, the memory is freed immediately.

```
import sys

a = [1, 2, 3]
print(sys.getrefcount(a))  # 2 (a + argument to getrefcount)

b = a
print(sys.getrefcount(a))  # 3 (a + b + argument)

del b
print(sys.getrefcount(a))  # 2 again
```

### 1.2.4 Garbage Collection Basics

Reference counting alone cannot handle circular references — where two objects reference each other.

```
a = {}
b = {}
a['ref'] = b
b['ref'] = a
# Both have refcount > 0 but are unreachable
# Python's cyclic GC detects and cleans these up
```

| | |
|---|---|
| **Interview** | Python uses reference counting as primary memory management. For circular references it cannot handle, Python uses a cyclic garbage collector that runs periodically. |

### 1.2.5 How Python Passes Arguments

Python is neither pass by value nor pass by reference. It is pass by object reference (also called pass by assignment).

```
# Mutable — function modifies the original
def modify_list(lst):
    lst.append(99)
```

```
nums = [1, 2, 3]
modify_list(nums)
print(nums)   # [1, 2, 3, 99]

# Immutable — reassignment doesn't affect original
def modify_int(n):
    n = n + 1      # creates a new local binding

x = 5
modify_int(x)
print(x)   # 5 — unchanged
```

| □ **Terms** | Pass by object reference · pass by assignment |
|---|---|

# 1.3 Python Performance Insights

## 1.3.1 List Internals

A Python list is not a simple array. It is a dynamic array — similar to ArrayList in Java or vector in C++. Python allocates more memory than needed when a list grows (over-allocation), which is why append() is O(1) amortized.

**Amortized O(1)** means: most appends are instant. Occasionally one append triggers a resize and copy of all elements, but averaged across all appends, the cost is still O(1) per operation.

| Operation | Time Complexity | Note |
|---|---|---|
| append() | O(1) amortized | Dynamic resize |
| pop() — from end | O(1) | Fast |
| pop(0) — from front | O(n) | □ Everything shifts left |
| insert(i, x) | O(n) | Shifts elements |
| index(x) | O(n) | Linear scan |
| in operator | O(n) | Linear scan |
| Slicing [i:j] | O(k) | k = slice length |

| □ **Trap** | list.pop(0) is O(n). Use collections.deque when you need O(1) removal from the front. |
|---|---|

### 1.3.2 Dict and Set Hashing

Python dict and set are both implemented as hash tables. When you do d[key], Python calls hash(key), uses the hash to find a bucket, and returns the value — all in O(1) average time.

```
# Only hashable (immutable) objects can be dict keys
d = {}
d[(1, 2)] = 'tuple key'     # valid — tuple is hashable
d[[1, 2]] = 'list key'      # TypeError — list is unhashable

# Membership test: set vs list
nums_list = list(range(10_000_000))
nums_set  = set(range(10_000_000))

999_999 in nums_list  # O(n) — scans entire list
999_999 in nums_set   # O(1) — hash lookup
```

| | |
|---|---|
| **Rule** | Always use a set for fast membership testing in interview problems. |

### 1.3.3 String Immutability and the Join Pattern

Strings are immutable. Every concatenation with + creates a brand new string object. In a loop with n iterations this is $O(n^2)$ total work — which causes TLE on large inputs.

```
# BAD — O(n²) total
result = ""
for i in range(n):
    result += str(i)   # new string every iteration

# GOOD — O(n) total
parts = []
for i in range(n):
    parts.append(str(i))
result = "".join(parts)   # single allocation at the end
```

### 1.3.4 Amortized Complexity in Practice

Amortized complexity means the average cost per operation over a sequence of operations — not the worst case of a single operation.

**Classic example:** Dynamic array resize. 99 appends cost O(1) each. 1 append triggers resize at O(n). Average across 100 operations = still O(1) per append.

**Applies to:** list.append() · hash table insertion · stack algorithms where each element is pushed and popped at most once.

| | |
|---|---|
| **Interview** | Say: "This is O(n) amortized because each element is processed at most twice — once on entry and once on exit from the stack." |

# 1.4 Python Built-in Tools for Algorithms

These are tools that experienced Python engineers use naturally. Knowing them makes your solutions cleaner, shorter, and faster to write in interviews.

## 1.4.1 enumerate and zip

```
# enumerate — index + value without manual counter
for i, val in enumerate(nums):
    print(i, val)

for i, val in enumerate(nums, start=1):  # start from 1
    print(i, val)

# zip — combine multiple iterables element by element
names  = ['Alice', 'Bob', 'Charlie']
scores = [90, 85, 92]
for name, score in zip(names, scores):
    print(name, score)

# Transpose a matrix
matrix     = [[1,2,3],[4,5,6],[7,8,9]]
transposed = list(zip(*matrix))
# [(1,4,7), (2,5,8), (3,6,9)]
```

## 1.4.2 lambda, sorted, min, max with key

```
# lambda creates a small anonymous function
square = lambda x: x * x

# Most useful as a key function in sorting
pairs = [(1,3),(2,1),(3,2)]
pairs.sort(key=lambda x: x[1])    # sort by second element
# [(2,1),(3,2),(1,3)]

words = ['banana','apple','cherry','date']
words.sort(key=lambda w: len(w))   # sort by length
```

```
students = [('Alice',90),('Bob',75),('Charlie',85)]
top = max(students, key=lambda s: s[1])   # ('Alice', 90)

# Sort descending — negate the key
pairs.sort(key=lambda x: -x[1])
```

### 1.4.3 List Comprehensions and Generator Expressions

List comprehensions are faster than equivalent for loops — they are optimised at the bytecode level.

```
# Basic comprehension
squares = [x * x for x in range(10)]

# With condition
evens = [x for x in range(20) if x % 2 == 0]

# Flatten 2D matrix
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [val for row in matrix for val in row]

# Dictionary comprehension
freq = {char: s.count(char) for char in set(s)}

# Set comprehension
unique_lengths = {len(word) for word in words}

# Generator expression — lazy, memory efficient
total = sum(x * x for x in range(10_000_000))  # no list created
```

### 1.4.4 defaultdict and Counter

**defaultdict** automatically creates a default value for missing keys — no more KeyError.

```
from collections import defaultdict

# Frequency counting
freq = defaultdict(int)
for char in 'hello':
    freq[char] += 1

# Grouping — common interview pattern (build adjacency list)
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)
```

**Counter** counts elements instantly and gives you powerful arithmetic methods.

```
from collections import Counter

s = 'mississippi'
freq = Counter(s)
# Counter({'s':4, 'i':4, 'p':2, 'm':1})

freq.most_common(2)   # [('s',4), ('i',4)]

# Counter arithmetic (very useful for anagram / sliding window)
c1 = Counter('aab')
c2 = Counter('abb')
c1 + c2   # Counter({'a':3,'b':3})
c1 - c2   # Counter({'a':1})
c1 & c2   # intersection: Counter({'a':1,'b':1})
c1 | c2   # union:        Counter({'a':2,'b':2})
```

### 1.4.5 deque from collections

A deque (double-ended queue) supports O(1) append and pop from both ends. A regular list is O(n) for operations at the front.

```
from collections import deque

q = deque()
q.append(1)       # O(1) — add to right
q.appendleft(0)   # O(1) — add to left
q.pop()           # O(1) — remove from right
q.popleft()       # O(1) — remove from left  (O(n) on a list!)

# maxlen — auto-discards oldest elements
window = deque(maxlen=3)
for num in [1,2,3,4,5]:
    window.append(num)
# deque([3,4,5], maxlen=3)
```

| | |
|---|---|
| **Use for** | BFS queue · Sliding window · Monotonic queue |

### 1.4.6 heapq module

Python's heapq implements a min-heap on top of a regular list. The smallest element is always at index 0.

```
import heapq

nums = [5,1,3,2,4]
heapq.heapify(nums)              # O(n) — convert list to heap
heapq.heappush(nums, 0)          # O(log n) — insert
smallest = heapq.heappop(nums)   # O(log n) — remove minimum
peek     = nums[0]               # O(1)     — view minimum

heapq.nlargest(3, nums)   # top 3 largest
heapq.nsmallest(3, nums)  # top 3 smallest

# Max-heap trick — negate values
max_heap = [-n for n in nums]
heapq.heapify(max_heap)
largest = -heapq.heappop(max_heap)

# Priority queue with tuples
heap = []
heapq.heappush(heap, (priority, value))
priority, value = heapq.heappop(heap)
```

### 1.4.7 bisect module

bisect performs binary search on a sorted list in O(log n) — without writing binary search yourself.

```
import bisect

nums = [1,3,5,7,9]

# bisect_left  — index of first element >= target
bisect.bisect_left(nums, 5)    # 2

# bisect_right — index of first element >  target
bisect.bisect_right(nums, 5)   # 3

# Insert while maintaining sorted order
bisect.insort(nums, 4)         # [1,3,4,5,7,9]

# Example — count elements less than target
count = bisect.bisect_left(nums, 5)   # 2 elements < 5

# bisect_left vs bisect_right on duplicates
nums = [1,3,5,5,5,7]
bisect.bisect_left(nums, 5)    # 2  (leftmost 5)
bisect.bisect_right(nums, 5)   # 5  (one past rightmost 5)
```

### 1.4.8 itertools

```
import itertools

# combinations — order doesn't matter
list(itertools.combinations([1,2,3], 2))
# [(1,2),(1,3),(2,3)]

# permutations — order matters
list(itertools.permutations([1,2,3], 2))
# [(1,2),(1,3),(2,1),(2,3),(3,1),(3,2)]

# product — cartesian product
list(itertools.product([0,1], repeat=3))  # all 3-bit binary strings

# accumulate — running cumulative sum
list(itertools.accumulate([1,2,3,4,5]))
# [1,3,6,10,15]

# groupby — group consecutive equal elements
for key, group in itertools.groupby('aaabbbcc'):
    print(key, list(group))
# a ['a','a','a']   b ['b','b','b']   c ['c','c']
```

### 1.4.9 functools — lru_cache and cache

lru_cache is Python's built-in memoization decorator. It automatically caches function results so repeated calls with the same arguments return instantly. This is the fastest way to write top-down dynamic programming in Python.

```
from functools import lru_cache, cache

# lru cache — classic memoization
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

print(fib(100))   # instant — each value computed once

# cache (Python 3.9+) — simpler, no size limit
@cache
def dp(i, j):
    # your 2D DP logic here
    pass

# functools.reduce — cumulative application
from functools import reduce
product = reduce(lambda a, b: a * b, [1,2,3,4,5])  # 120
```

# 1.5 Common Python Pitfalls in Interviews

These are bugs that experienced developers recognise immediately but beginners fall into repeatedly. Knowing these makes you look sharp in interviews.

## 1.5.1 Mutable Default Arguments

One of the most famous Python bugs. Default arguments are evaluated once when the function is defined — not each time it is called.

```
# WRONG — the list is created ONCE and shared across all calls
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item(1))   # [1]
print(add_item(2))   # [1, 2]  — unexpected!
print(add_item(3))   # [1, 2, 3] — unexpected!

# CORRECT — use None and create inside the function
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
```

## 1.5.2 Shallow vs Deep Copy

```
import copy

original = [[1,2],[3,4]]

# Assignment — same object
ref = original
ref[0].append(99)
print(original)   # [[1,2,99],[3,4]] — modified!

# Shallow copy — new outer list, same inner lists
shallow = original.copy()   # or list(original) or original[:]
shallow[0].append(88)
print(original)   # [[1,2,99,88],[3,4]] — still modified!

# Deep copy — completely independent
deep = copy.deepcopy(original)
deep[0].append(77)
print(original)   # [[1,2,99,88],[3,4]] — NOT modified
```

| | |
|---|---|
| **Rule** | When copying a 2D grid or nested structure in DP or backtracking, use copy.deepcopy() or reconstruct manually. |

### 1.5.3 Integer Overflow? Python Does Not Have It

In C++ and Java, integers overflow when they exceed the maximum value for their type (e.g. 2^31 - 1 for 32-bit int). Python integers have arbitrary precision — they never overflow. Python automatically allocates more memory as numbers grow.

```
print(2 ** 1000)   # works perfectly — a huge number, no overflow
```

| | |
|---|---|
| **Interview** | Mention this to interviewers when a problem involves large numbers. It is a real advantage of Python. |

### 1.5.4 Floating Point Comparison Issues

```
print(0.1 + 0.2 == 0.3)   # False!
print(0.1 + 0.2)           # 0.30000000000000004

# Safe comparison
import math
math.isclose(0.1 + 0.2, 0.3)      # True
abs(0.1 + 0.2 - 0.3) < 1e-9       # True
```

| | |
|---|---|
| **Note** | This is not a Python bug — it is how IEEE 754 floating point works in every language including C, Java, and JavaScript. |

### 1.5.5 List Multiplication Trap — 2D Grid Initialisation

This is one of the most common bugs in 2D array initialisation in coding interviews.

```
# WRONG — n references to the SAME inner list
grid = [[0] * 3] * 3
grid[0][0] = 1
print(grid)
# [[1,0,0],[1,0,0],[1,0,0]]  — all rows changed!

# CORRECT — independent inner lists
grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 1
```

```
print(grid)
# [[1,0,0],[0,0,0],[0,0,0]]  — only row 0 changed
```

| | |
|---|---|
| **Rule** | Always initialise 2D grids with list comprehension. Never use * multiplication for nested lists. |

# Section 1 — Quick Revision Cheat Sheet

| Topic | Key Fact |
|---|---|
| Python Execution | source → bytecode → PVM (CPython) |
| Call Stack | each function call = one stack frame, limit = 1000 |
| Stack vs Heap | variables on stack, objects on heap |
| Mutable | list, dict, set — changed in place |
| Immutable | int, str, tuple — new object on change |
| Argument passing | pass by object reference |
| list.append() | O(1) amortized — dynamic array |
| dict/set lookup | O(1) average — hash table |
| String concat loop | use "".join() not += |
| lru_cache / cache | instant memoization for top-down DP |
| deque | O(1) both ends — use for BFS and sliding window |
| heapq | min-heap by default, negate for max-heap |
| bisect | binary search on sorted list in O(log n) |
| 2D grid init | [[0]*n for _ in range(m)]  — NOT [[0]*n]*m |
| Integer overflow | does not exist in Python — arbitrary precision |
| is vs == | use == for values, is only for None checks |

**End of Section 1.** Continue with Section 2 — Data Structures Cheat Sheet.

# Python & Algorithms for Coding Interviews

Section 2 — Data Structures Cheat Sheet

## 2.1 Arrays / Lists

### What It Is

An array is a contiguous block of memory storing elements of the same type. Python's list is a dynamic array — it automatically resizes as you add elements. It is the most fundamental and frequently used data structure in coding interviews.

### Intuition

Think of a list as a numbered row of boxes. You can instantly reach any box by its number (index). Adding to the end is fast. Inserting or removing from the middle requires shifting all boxes after that point.

### Analogy

| | |
|---|---|
| **Mental Model** | A row of lockers at school. Each locker has a number. You can open locker 5 directly without checking lockers 1–4. But if you insert a new locker in the middle, every locker after it must be renumbered and moved. |

### Internal Working

Python lists are implemented as arrays of pointers (object references). The list object stores a pointer to a block of memory containing pointers to each element. When the list runs out of space, Python allocates a larger block (roughly 1.125× the current size) and copies all pointers over. This is why append() is O(1) amortized but occasional appends cost O(n).

## Python Usage

Python lists are used for almost everything — storing results, managing DFS/BFS queues, prefix sums, frequency arrays, sliding windows, and DP tables.

## Code Example

```
nums = [1, 2, 3, 4, 5]

# Access by index — O(1)
print(nums[0])    # 1
print(nums[-1])   # 5  (last element)

# Slicing — O(k) where k = slice length
print(nums[1:4])  # [2, 3, 4]

# Append — O(1) amortized
nums.append(6)

# Insert — O(n)
nums.insert(2, 99)   # insert 99 at index 2

# Remove — O(n)
nums.remove(99)      # removes first occurrence

# Pop from end — O(1)
nums.pop()

# Pop from front — O(n) □ use deque instead
nums.pop(0)

# Check membership — O(n)
3 in nums

# 2D array (grid) — correct initialisation
grid = [[0] * cols for _ in range(rows)]
```

## Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| Access by index | O(1) | O(n) |
| Append (end) | O(1) amortized | |
| Pop (end) | O(1) | |
| Pop (front) / insert | O(n) | |

| | | |
|---|---|---|
| Search (in) | O(n) | |
| Slice [i:j] | O(k) | O(k) |
| Sort | O(n log n) | O(n) |

### Interview Use Cases

- Two pointer problems — left/right pointers on same array
- Sliding window — maintain a window over contiguous elements
- Prefix sums — precompute cumulative values
- DP tables — store subproblem results
- BFS/DFS — store adjacency lists
- Frequency arrays — count characters or numbers

### Real World Python Usage

| | |
|---|---|
| **Real World** | Django ORM querysets return lists. NumPy arrays underpin all ML model training. FastAPI response bodies are often lists of Pydantic objects. Pandas DataFrames are built on NumPy arrays internally. |

## 2.2 Hash Tables (dict and set)

### What It Is

A hash table maps keys to values using a hash function. Python's dict and set are both hash tables. They provide O(1) average-case lookup, insertion, and deletion — making them the most powerful tool in interview problem solving.

### Intuition

Imagine you have a million books and you want to find one instantly. Instead of searching shelf by shelf, you compute a 'code' from the book title and go directly to that shelf. The code is the hash, the shelf is the bucket.

### Analogy

| | |
|---|---|
| **Mental Model** | A library with a magic formula. Give any book title to the formula and it tells you exactly which shelf it is on — no searching needed. Hash tables work the same way. |

## Internal Working

Python's dict uses open addressing with a hash table. When you set d[key] = value, Python calls hash(key) to compute an integer index. It places the value at that index in an internal array. On lookup, the same hash is computed and the value is retrieved in O(1). Collisions (two keys hashing to the same index) are resolved by probing nearby slots.

## Python Usage

Use dict for key-value mapping, counting, memoization, and graph adjacency. Use set for O(1) membership testing and deduplication.

## Code Example

```
from collections import defaultdict, Counter

# Basic dict operations
d = {}
d['key'] = 'value'          # insert — O(1)
val = d.get('key', None)    # safe get with default
del d['key']                # delete — O(1)
'key' in d                  # membership — O(1)

# defaultdict — no KeyError for missing keys
freq = defaultdict(int)
for ch in 'hello':
    freq[ch] += 1

# Counter — instant frequency map
freq = Counter('mississippi')
# Counter({'s':4,'i':4,'p':2,'m':1})

# Set operations
seen = set()
seen.add(5)           # O(1)
5 in seen             # O(1)
seen.discard(5)       # O(1) — no error if missing

# Set math
a = {1, 2, 3}
b = {2, 3, 4}
a & b   # intersection: {2, 3}
a | b   # union:        {1, 2, 3, 4}
a - b   # difference:   {1}
```

### Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| Insert / Update | O(1) average | O(n) |
| Lookup / Delete | O(1) average | |
| Membership test | O(1) average | |
| Iteration | O(n) | |
| Worst case (collisions) | O(n) | |

### Interview Use Cases

- Frequency counting — count characters, words, or numbers
- Two Sum pattern — store complement values for O(1) lookup
- Memoization — cache recursive call results
- Grouping — group anagrams, group by property
- Graph adjacency list — defaultdict(list)
- Sliding window — track character counts in a window

### Real World Python Usage

| | |
|---|---|
| **Real World** | Python dict is used in caching systems (Redis-like in-memory caches), session storage in Django/Flask, JSON parsing (every JSON object becomes a dict), and configuration management. Sets are used in deduplication pipelines and bloom filter approximations. |

## 2.3 Stacks

### What It Is

A stack is a Last In First Out (LIFO) data structure. The last element you push onto the stack is the first one you pop off. Python implements stacks using a regular list — push with append(), pop with pop().

### Intuition

Think of undo history in a text editor. Every action gets added to the top. When you press Ctrl+Z, the most recent action is undone first. You always interact with the top of the stack.

## Analogy

| | |
|---|---|
| **Mental Model** | A stack of dinner plates. You always place a new plate on top and always take from the top. You cannot take a plate from the middle without removing all the plates above it first. |

## Internal Working

A stack is conceptually simple — it is just a list where you only ever touch the right end. append() adds to the top (O(1)). pop() removes from the top (O(1)). Peeking at the top is done with stack[-1] — O(1). Python's list is perfect for this since append and pop from the right end are both O(1).

## Python Usage

Python list used as stack: append() to push, pop() to pop, stack[-1] to peek.

## Code Example

```
# Stack using Python list
stack = []

stack.append(1)     # push — O(1)
stack.append(2)
stack.append(3)

top = stack[-1]     # peek — O(1), no removal
val = stack.pop()   # pop  — O(1)

if not stack:       # check empty
    print('empty')

# Classic pattern — valid parentheses
def is_valid(s):
    stack = []
    pairs = {')':'(', ']':'[', '}':'{'}
    for ch in s:
        if ch in '([{':
            stack.append(ch)
        elif stack and stack[-1] == pairs[ch]:
            stack.pop()
        else:
            return False
```

```
    return len(stack) == 0
```

### Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| Push (append) | O(1) | O(n) |
| Pop | O(1) | |
| Peek (top) | O(1) | |
| Search | O(n) | |

### Interview Use Cases

- Balanced parentheses — valid brackets problems
- Monotonic stack — next greater / smaller element
- DFS iterative — explicit stack replaces call stack
- Expression evaluation — infix to postfix, calculator
- Backtracking — undo moves in iterative solutions
- Call stack simulation — flatten nested structures

### Real World Python Usage

| | |
|---|---|
| **Real World** | Python's own call stack is a stack (as covered in Section 1). Django middleware uses a stack-based execution chain. Undo/redo systems in editors use two stacks. Browser history is a stack. Python's contextlib uses a stack-based context manager. |

## 2.4 Queues

### What It Is

A queue is a First In First Out (FIFO) data structure. The first element you enqueue is the first one you dequeue. Queues are fundamental to BFS algorithms. In Python, always use collections.deque — never a list — for queue operations.

### Intuition

Think of a queue as a line at a ticket counter. The first person in line is served first. New people join at the back. Nobody cuts the line.

## Analogy

| | |
|---|---|
| **Mental Model** | A queue at a coffee shop. The first customer in line gets their coffee first. You always add new customers at the back and serve from the front. |

## Internal Working

A deque (double-ended queue) is implemented as a doubly-linked list of fixed-size blocks. This allows O(1) append and pop from both ends. A regular Python list has O(n) for pop(0) because all elements must shift left — never use list as a queue in interviews.

## Python Usage

Use collections.deque for all queue operations. It supports appendleft(), append(), popleft(), pop() — all O(1).

## Code Example

```
from collections import deque

queue = deque()

queue.append(1)      # enqueue — O(1)
queue.append(2)
queue.append(3)

front = queue[0]         # peek front — O(1)
val   = queue.popleft() # dequeue   — O(1)

# BFS template using queue
def bfs(graph, start):
    visited = set([start])
    queue   = deque([start])
    while queue:
        node = queue.popleft()
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
```

### Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| Enqueue (append) | O(1) | O(n) |
| Dequeue (popleft) | O(1) | |
| Peek front | O(1) | |
| list.pop(0) □ | O(n) | |

### Interview Use Cases

- BFS — level-by-level tree and graph traversal
- Shortest path in unweighted graphs
- Multi-source BFS — multiple starting nodes simultaneously
- Level order tree traversal
- Sliding window using deque — monotonic queue
- Task scheduling simulation

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Python's asyncio event loop processes coroutines using a queue. Django Channels uses a message queue for WebSocket handling. Celery task queues distribute background jobs. Print spoolers and OS process schedulers use FIFO queues. |

## 2.5 Deque (Double-Ended Queue)

### What It Is

A deque allows O(1) insertions and deletions from both the front and the back. It is more powerful than both a stack and a queue, which only support one-ended operations. In Python, collections.deque is the standard implementation.

### Intuition

A deque is like a double-ended tube. You can push and pop elements from either end with equal speed. This makes it perfect for algorithms that need both stack and queue behaviour — like the sliding window maximum.

## Analogy

| | |
|---|---|
| □ **Mental Model** | A train where passengers can board and exit from both the front and the rear door simultaneously — and both doors are equally fast. |

## Internal Working

CPython's deque is implemented as a doubly-linked list of fixed-size blocks (each block holds 64 elements). This gives O(1) operations at both ends without the need to shift elements. Random access in the middle (deque[i]) is O(n) — unlike lists which are O(1). Use deque when you need both ends, list when you need random access.

## Python Usage

collections.deque with appendleft(), append(), popleft(), pop(). Use maxlen parameter to auto-discard old elements.

## Code Example

```
from collections import deque

dq = deque([1, 2, 3])

dq.append(4)        # add to right  — O(1)
dq.appendleft(0)    # add to left   — O(1)
dq.pop()            # remove right  — O(1)
dq.popleft()        # remove left   — O(1)

dq[0]               # peek left     — O(1)
dq[-1]              # peek right    — O(1)
dq[2]               # random access — O(n) □

# maxlen — fixed-size sliding window
window = deque(maxlen=3)
for num in [1, 2, 3, 4, 5]:
    window.append(num)
# deque([3, 4, 5], maxlen=3)

# Monotonic deque — sliding window maximum
def max_sliding_window(nums, k):
    dq, result = deque(), []
    for i, num in enumerate(nums):
```

```
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] < i - k + 1:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

### Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| append / appendleft | O(1) | O(n) |
| pop / popleft | O(1) | |
| Peek both ends | O(1) | |
| Random access [i] | O(n) | |

### Interview Use Cases

- Sliding window maximum — monotonic deque
- BFS queue — popleft is O(1)
- Palindrome check — compare from both ends
- LRU Cache — deque tracks usage order
- Sliding window problems needing index tracking

### Real World Python Usage

| | |
|---|---|
| **Real World** | Python's own asyncio uses deque internally for task scheduling. The LRU cache in functools is implemented with an ordered dict (which internally behaves like a deque). Browser tab history can use a deque structure. |

## 2.6 Heaps (Priority Queues)

### What It Is

A heap is a complete binary tree that satisfies the heap property: in a min-heap, every parent is smaller than or equal to its children. The minimum element is always at the root — accessible in O(1). Insertion and removal are O(log n). Python's heapq implements a min-heap.

## Intuition

A heap is like a tournament bracket where the winner (smallest element) always bubbles to the top. Every time you remove the top winner, the bracket reorganises to find the next winner in O(log n) time.

## Analogy

| | |
|---|---|
| **Mental Model** | An airport boarding queue. First class passengers (highest priority = smallest number) always board first, no matter when they arrived. When a first class passenger boards, the next highest priority passenger immediately moves to the front. |

## Internal Working

A heap is stored as a flat array. For a node at index i, its left child is at 2i+1, right child at 2i+2, and parent at (i-1)//2. When you push a new element, it goes to the end (O(1)) and bubbles up (sifts up) by swapping with its parent until the heap property is restored — O(log n). When you pop the root, the last element replaces it and sifts down — O(log n).

## Python Usage

heapq module. Python only provides min-heap. For max-heap, negate values when pushing and negate again when popping.

## Code Example

```
import heapq

# Min-heap
heap = []
heapq.heappush(heap, 5)      # O(log n)
heapq.heappush(heap, 1)
heapq.heappush(heap, 3)

smallest = heap[0]           # peek   — O(1)
val = heapq.heappop(heap)    # pop    — O(log n) → returns 1

nums = [5, 1, 3, 2, 4]
heapq.heapify(nums)          # convert list in-place — O(n)
```

```
# Max-heap — negate values
max_heap = []
heapq.heappush(max_heap, -5)
heapq.heappush(max_heap, -1)
largest = -heapq.heappop(max_heap)   # 5

# Heap with (priority, value) tuples
tasks = []
heapq.heappush(tasks, (1, 'high priority'))
heapq.heappush(tasks, (3, 'low priority'))
heapq.heappush(tasks, (2, 'medium priority'))
priority, task = heapq.heappop(tasks)   # (1, 'high priority')

# K largest elements
heapq.nlargest(3, nums)    # O(n log k)
heapq.nsmallest(3, nums)   # O(n log k)
```

## Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| heappush | O(log n) | O(n) |
| heappop (min) | O(log n) | |
| Peek minimum | O(1) | |
| heapify | O(n) | |
| nlargest / nsmallest | O(n log k) | |

## Interview Use Cases

- Top K elements — K largest, K smallest, K most frequent
- Kth largest / smallest element
- Merge K sorted lists
- Dijkstra shortest path — min-heap on distances
- Task scheduling — process jobs by priority
- Median of data stream — two heaps technique

## Real World Python Usage

| | |
|---|---|
| **Real World** | Python's heapq powers task schedulers in asyncio. Celery uses priority queues for task dispatch. Database query optimisers use heap structures for top-k result processing. Operating system process schedulers use priority queues. |

# 2.7 Linked Lists

## What It Is

A linked list is a sequence of nodes where each node stores a value and a pointer to the next node. Unlike arrays, nodes are not stored contiguously in memory — each node lives wherever Python allocates it. Python has no built-in linked list, so you implement it with a Node class.

## Intuition

A linked list is like a treasure hunt where each clue tells you the location of the next clue. You cannot skip ahead — you must follow the chain from the beginning. But inserting a new clue anywhere in the chain only requires updating two pointers, not moving everything else.

## Analogy

| | |
|---|---|
| **Mental Model** | A chain of train carriages. Each carriage is connected to the next. To add a new carriage in the middle, you just disconnect two carriages and reconnect with the new one in between — no other carriages need to move. |

## Internal Working

Each node holds data and a next pointer. The head pointer points to the first node. The last node's next pointer is None. Singly linked lists only have next. Doubly linked lists have both next and prev, enabling O(1) deletion when you have a reference to the node. Python's collections.deque is actually a doubly linked list of arrays internally.

## Python Usage

Implement with a custom Node class. Python's deque is backed by a linked list. Use dummy head nodes to simplify edge cases.

## Code Example

```
# Node definition
class ListNode:
    def __init__(self, val=0, next=None):
        self.val  = val
        self.next = next
```

```
# Build:  1 -> 2 -> 3 -> None
head = ListNode(1)
head.next = ListNode(2)
head.next.next = ListNode(3)

# Traverse
node = head
while node:
    print(node.val)
    node = node.next

# Reverse a linked list — classic interview problem
def reverse(head):
    prev, curr = None, head
    while curr:
        nxt       = curr.next
        curr.next = prev
        prev      = curr
        curr      = nxt
    return prev

# Dummy head — simplifies insertion/deletion edge cases
dummy = ListNode(0)
dummy.next = head

# Fast and slow pointer — find middle
def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

## Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| Access by index | O(n) | O(n) |
| Insert at head | O(1) | |
| Insert at tail | O(n) / O(1) with tail ptr | |
| Insert at position | O(n) | |
| Delete (with node ref) | O(1) doubly linked | |
| Search | O(n) | |

## Interview Use Cases

- Reverse a linked list — iterative and recursive

- Detect cycle — fast and slow pointers
- Find middle node — fast and slow pointers
- Merge two sorted lists
- Remove Nth node from end
- LRU Cache — doubly linked list + hash map

### Real World Python Usage

| | |
|---|---|
| **Real World** | Python's collections.deque is a doubly linked list internally. LRU cache implementations use a doubly linked list paired with a hash map. Memory allocators use free-list linked lists. File systems use linked structures for block chains. |

## 2.8 Trees

### What It Is

A tree is a hierarchical data structure with a root node at the top. Every node has zero or more child nodes. A binary tree is the most common type in interviews — each node has at most two children (left and right). Trees are acyclic, connected graphs with a designated root.

### Intuition

A tree is like a company org chart. The CEO is the root. Each manager (node) has direct reports (children). Leaf nodes are employees with no direct reports. The depth of a node is how many levels it is from the root. The height of the tree is the longest path from root to any leaf.

### Analogy

| | |
|---|---|
| **Mental Model** | A family tree. The great-grandparent is the root. Their children, grandchildren, and great-grandchildren form branches. Leaf nodes are the youngest generation with no children. |

### Internal Working

Trees are implemented with nodes, each containing a value and references to children. Binary trees have left and right child references. The height of a balanced binary tree with n nodes is

O(log n). A skewed (unbalanced) tree can degrade to height O(n) — effectively a linked list. DFS and BFS are the two fundamental ways to traverse a tree.

## Python Usage

Implement with TreeNode class. Use recursion for DFS traversals. Use deque for level-order BFS traversal.

## Code Example

```
# TreeNode definition (standard in LeetCode)
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val   = val
        self.left  = left
        self.right = right

# Inorder:   Left → Root → Right  (gives sorted order in BST)
def inorder(root):
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)

# Preorder:  Root → Left → Right  (good for copying a tree)
def preorder(root):
    if not root: return []
    return [root.val] + preorder(root.left) + preorder(root.right)

# Postorder: Left → Right → Root  (good for deleting a tree)
def postorder(root):
    if not root: return []
    return postorder(root.left) + postorder(root.right) + [root.val]

# Level-order BFS traversal
from collections import deque
def level_order(root):
    if not root: return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:  queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result

# Tree height
def height(root):
    if not root: return 0
    return 1 + max(height(root.left), height(root.right))
```

### Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| DFS traversal | O(n) | O(h) stack |
| BFS traversal | O(n) | O(w) queue |
| Height / depth | O(n) | O(h) |
| h = balanced tree | O(log n) | |
| h = skewed tree | O(n) | |

### Interview Use Cases

- Tree traversals — inorder, preorder, postorder, level-order
- Lowest Common Ancestor (LCA)
- Maximum path sum
- Diameter of binary tree
- Symmetric / mirror tree check
- Serialise and deserialise a tree

### Real World Python Usage

| | |
|---|---|
| **Real World** | File systems are trees — directories are internal nodes, files are leaves. HTML/XML DOM is a tree structure. Python's AST (Abstract Syntax Tree) represents your code as a tree. Decision trees in machine learning are literal binary trees. |

## 2.9 Binary Search Trees (BST)

### What It Is

A Binary Search Tree is a binary tree with a key property: for every node, all values in the left subtree are smaller, and all values in the right subtree are larger. This property enables O(log n) search, insert, and delete in a balanced tree.

### Intuition

A BST is like a binary decision game. At each node, you ask: is my target smaller or larger than this value? If smaller, go left. If larger, go right. Each step eliminates half the remaining possibilities — just like binary search on a sorted array.

## Analogy

| | |
|---|---|
| **Mental Model** | A library sorted alphabetically on shelves arranged in a tree. At the middle shelf you find 'M'. Searching for 'Apple'? Go left. Searching for 'Zebra'? Go right. Each decision halves your search space. |

## Internal Working

The BST property makes inorder traversal produce a sorted sequence — this is the most important property to remember. A balanced BST (like AVL tree or Red-Black tree) guarantees O(log n) height. Python has no built-in BST. The sortedcontainers library provides SortedList. In interviews, you implement BST logic manually.

## Python Usage

Implement manually in interviews. Use sortedcontainers.SortedList for production code. Inorder traversal of BST gives sorted output.

## Code Example

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

# Search — O(log n) balanced, O(n) worst
def search(root, target):
    if not root or root.val == target:
        return root
    if target < root.val:
        return search(root.left, target)
    return search(root.right, target)

# Insert — O(log n) balanced
def insert(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left  = insert(root.left, val)
    else:
        root.right = insert(root.right, val)
    return root
```

```
# Validate BST — pass min/max bounds
def is_valid_bst(root, low=float('-inf'), high=float('inf')):
    if not root: return True
    if not (low < root.val < high): return False
    return (is_valid_bst(root.left,  low, root.val) and
            is_valid_bst(root.right, root.val, high))

# Inorder traversal gives sorted sequence
def inorder(root):
    if not root: return []
    return inorder(root.left) + [root.val] + inorder(root.right)
```

### Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| Search / Insert / Delete (balanced) | O(log n) | O(h) |
| Search / Insert / Delete (skewed) | O(n) | |
| Inorder traversal | O(n) | O(h) |
| Min / Max | O(h) | |

### Interview Use Cases

- Validate a BST — check bounds recursively
- Find Kth smallest/largest element in BST
- LCA in a BST
- Convert sorted array to balanced BST
- BST iterator
- Range sum query — sum all nodes in [L, R]

### Real World Python Usage

| | |
|---|---|
| **Real World** | Database indexes (B-trees, which are generalised BSTs) power every SQL query. Python's sortedcontainers.SortedList is used in competitive programming and data science. Git commit history is a tree. Event-driven schedulers use BST-like structures for time-ordered event dispatch. |

## 2.10 Graphs

## What It Is

A graph is a collection of nodes (vertices) connected by edges. Unlike trees, graphs can have cycles, multiple paths between nodes, and disconnected components. Graphs model real-world networks: roads, social connections, dependencies, and more. They are the most complex and most rewarding data structure in interviews.

## Intuition

A graph is like a city road map. Every intersection is a node. Every road is an edge. A directed graph (digraph) means roads can be one-way. A weighted graph means roads have distances. BFS finds the shortest path. DFS explores all paths.

## Analogy

| | |
|---|---|
| **Mental Model** | A city metro map. Each station is a node. Each track between stations is an edge. Weighted edges represent travel time. Directed edges represent one-way routes. Finding the fastest route from A to B is shortest path. Checking all reachable stations from A is graph traversal. |

## Internal Working

Graphs are stored as adjacency lists (dict of lists) for sparse graphs or adjacency matrices (2D array) for dense graphs. Adjacency lists use O(V + E) space — far more efficient than adjacency matrices ($O(V^2)$) for most real-world graphs. BFS uses a queue and explores level by level. DFS uses a stack (or recursion) and explores as deep as possible before backtracking.

## Python Usage

Use defaultdict(list) for adjacency list. Use a set for visited nodes. BFS with deque, DFS with recursion or explicit stack.

## Code Example

```
from collections import defaultdict, deque

# Build adjacency list — undirected graph
graph = defaultdict(list)
edges = [(0,1),(0,2),(1,3),(2,3),(3,4)]
for u, v in edges:
    graph[u].append(v)
```

```
    graph[v].append(u)   # omit for directed graph

# BFS — shortest path, level-order
def bfs(start, target):
    visited = set([start])
    queue   = deque([(start, 0)])   # (node, distance)
    while queue:
        node, dist = queue.popleft()
        if node == target: return dist
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, dist + 1))
    return -1

# DFS — explore all paths, detect cycles
def dfs(node, visited):
    visited.add(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs(nb, visited)

# Count connected components
def count_components(n, edges):
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
    visited, count = set(), 0
    for node in range(n):
        if node not in visited:
            dfs(node, visited)
            count += 1
    return count
```

## Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| BFS / DFS traversal | O(V + E) | O(V) |
| Adjacency list space | | O(V + E) |
| Adjacency matrix space | | $O(V^2)$ |
| Edge lookup (adj list) | O(degree) | |
| Edge lookup (adj matrix) | O(1) | |

## Interview Use Cases

- Number of islands — DFS/BFS on a matrix grid
- Shortest path in unweighted graph — BFS

- Detect cycle — directed and undirected
- Connected components — DFS/Union Find
- Bipartite graph check — BFS colouring
- Clone a graph

### Real World Python Usage

| | |
|---|---|
| **Real World** | Google Maps routing is graph shortest-path. Social network friend suggestions use graph BFS. Python's import system resolves module dependencies using a DAG. RAG pipelines in AI use knowledge graphs. Network packet routing uses graph algorithms. |

## 2.11 Tries (Prefix Trees)

### What It Is

A trie (pronounced 'try') is a tree-shaped data structure optimised for storing and searching strings by their prefixes. Each node represents one character. A path from the root to a node spells out a prefix. A path from root to an end node spells a complete word. Insert, search, and prefix-search all run in O(L) time where L is the length of the word.

### Intuition

A trie is like an autocomplete system. When you type 'app' into Google, it instantly shows 'apple', 'application', 'appointment'. All words sharing the prefix 'app' share the same branch of the trie — no repeated character storage.

### Analogy

| | |
|---|---|
| **Mental Model** | A phone keypad word tree. Every time you press a letter, you go one level deeper in the tree. All words that share the same starting letters share the same path from the root. The trie 'fans out' as words diverge. |

### Internal Working

Each TrieNode stores a dictionary of children (one per character) and an is_end flag marking the end of a complete word. Inserting a word of length L visits or creates L nodes — O(L).

Searching works the same way. The total space is O(ALPHABET_SIZE × N × L) in the worst case, but shared prefixes save significant space in practice.

## Python Usage

Implement with a TrieNode class containing a children dict and is_end boolean flag.

## Code Example

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end   = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_end

    def starts_with(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return True

# Usage
trie = Trie()
trie.insert('apple')
trie.search('apple')      # True
trie.search('app')        # False — not a complete word
trie.starts_with('app')   # True — valid prefix
```

## Time & Space Complexity

| Operation | Time | Space |
|---|---|---|
| Insert | O(L) | O(L) |
| Search | O(L) | O(1) |
| Prefix search | O(L) | O(1) |
| Space (all words) | O(ALPHABET × N × L) | |

### Interview Use Cases

- Implement Trie — insert, search, startsWith
- Word Search II — find all words in a grid using trie
- Replace words — replace words with their root prefix
- Design search autocomplete system
- Longest word in dictionary
- Map sum pairs — sum values of keys with given prefix

### Real World Python Usage

| | |
|---|---|
| **Real World** | Search engines use tries for autocomplete suggestions. IDEs use tries for code completion. IP routing tables use a form of trie (Patricia trie) for prefix matching. Spell checkers use tries to find similar words. DNS resolution uses trie-like prefix matching. |

## 2.12 Data Structure Comparison Table

Use this table to quickly decide which data structure to use when you read a problem.

| Data Structure | Best For | Key Operation | Python |
|---|---|---|---|
| Array / List | Indexed access, iteration | O(1) index access | list |
| Hash Table | Key-value, counting, fast lookup | O(1) get/set | dict, set |
| Stack | LIFO, undo, DFS, brackets | O(1) push/pop | list (append/pop) |
| Queue | FIFO, BFS, level-order | O(1) enqueue/dequeue | deque |
| Deque | Both ends, sliding window | O(1) both ends | collections.deque |

| | | | |
|---|---|---|---|
| Heap | Top K, priority, scheduling | O(log n) push/pop | heapq |
| Linked List | Insert/delete middle, LRU | O(1) insert with ref | custom Node class |
| Tree | Hierarchical data, traversals | O(h) traversal | custom TreeNode |
| BST | Sorted data, range queries | O(log n) search | sortedcontainers |
| Graph | Networks, paths, components | O(V+E) BFS/DFS | defaultdict(list) |
| Trie | Prefix search, autocomplete | O(L) insert/search | custom TrieNode |

# Section 2 — Quick Revision Cheat Sheet

| Data Structure | Key Fact to Remember |
|---|---|
| List | Dynamic array — O(1) append, O(n) insert/delete middle, O(n) pop(0) |
| dict / set | Hash table — O(1) avg get/set/delete/in. Only hashable keys. |
| Stack | LIFO — list.append() push, list.pop() pop, list[-1] peek |
| Queue | FIFO — always use deque, never list (pop(0) is O(n)) |
| Deque | O(1) both ends — use for BFS, sliding window, monotonic queue |
| Heap | Min-heap via heapq — negate values for max-heap |
| Linked List | O(1) insert at head — use dummy node to simplify edge cases |
| Tree | Inorder=sorted(BST), BFS=level order, DFS=recursion |
| BST | Inorder gives sorted output — validate with min/max bounds |
| Graph | defaultdict(list) for adjacency list — BFS=shortest path, DFS=explore all |
| Trie | children dict + is_end flag — O(L) for all ops where L = word length |

**End of Section 2.** Continue with Section 3 — Core Algorithmic Patterns.

# Python & Algorithms for Coding Interviews

Section 3A — Core Algorithmic Patterns (3.1 – 3.10)

## 3.1 Two Pointers

### The Problem Situation

You are given a sorted array (or string) and need to find a pair, triplet, or subarray satisfying some condition. A brute force nested loop would be $O(n^2)$. Two pointers reduce this to $O(n)$ by moving a left and right pointer intelligently toward each other.

### Key Insight

| | |
|---|---|
| **Insight** | When the array is sorted, you can place one pointer at the start and one at the end. If their combined value is too large, shrink from the right. If too small, grow from the left. Each step moves at least one pointer, so the total work is $O(n)$. |

### Analogy

| | |
|---|---|
| **Mental Model** | Two people walking toward each other on a number line. If together they overshoot the target, the person on the right takes a step left. If they undershoot, the person on the left takes a step right. They never pass each other and they always converge in $O(n)$ steps. |

### Basic Code Template

```
# Two pointers — opposite ends (sorted array)
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if total == target:
            return [left, right]
        elif total < target:
```

```
            left  += 1   # need larger sum
        else:
            right -= 1   # need smaller sum
    return []
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 167 — Two Sum II • LeetCode 15 — 3Sum • LeetCode 11 — Container With Most Water |

## Alternative Template

```
# Two pointers — same direction (partition / remove)
def remove_duplicates(nums):
    if not nums: return 0
    slow = 0
    for fast in range(1, len(nums)):
        if nums[fast] != nums[slow]:
            slow += 1
            nums[slow] = nums[fast]
    return slow + 1
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 26 — Remove Duplicates • LeetCode 27 — Remove Element • LeetCode 977 — Squares of Sorted Array |

## Example Walkthrough

```
# Example: LeetCode 15 — 3Sum
# Find all unique triplets that sum to 0
def three_sum(nums):
    nums.sort()
    result = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i-1]:  # skip duplicates
            continue
        left, right = i + 1, len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total == 0:
                result.append([nums[i], nums[left], nums[right]])
                while left < right and nums[left]  == nums[left+1]:  left
+= 1
                while left < right and nums[right] == nums[right-1]: right
-= 1
                left += 1; right -= 1
            elif total < 0: left  += 1
            else:           right -= 1
    return result

# Trace: nums = [-4,-1,-1,0,1,2]
# i=1 (val=-1): left=2,right=5 → -1+(-1)+2=0 ✓ → append
# i=1 (val=-1): left=3,right=4 → -1+0+1=0  ✓ → append
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Typical | O(n) | O(1) |
| With sort | O(n log n) | O(1) |

## Pattern Recognition Clues

- Keywords: sorted array/string, pair/triplet, sum/difference target
- Input is sorted OR can be sorted first without changing the answer
- Problem asks for a pair or asks to partition an array in place
- $O(n^2)$ brute force is obvious — two pointers gives O(n)

## Interview Traps

| | |
|---|---|
| □ **Trap** | Forgetting to sort first — two pointers only work reliably on sorted arrays |
| □ **Trap** | Not skipping duplicate values in 3Sum — leads to duplicate triplets in result |
| □ **Trap** | Using left <= right instead of left < right — processes same element twice |

## Edge Cases

- Empty array or single element
- All elements are the same
- No valid pair exists — must return empty or -1
- Negative numbers in sum problems

## Important Practice Problems

- LeetCode 167 — Two Sum II - Input Array Is Sorted
- LeetCode 15 — 3Sum
- LeetCode 11 — Container With Most Water
- LeetCode 42 — Trapping Rain Water
- LeetCode 977 — Squares of a Sorted Array

## Real World Python Usage

| | |
|---|---|
| □ **Real World** | Two pointers are used in merge operations inside Merge Sort. Database JOIN operations on sorted columns use a similar merge pointer technique. Network packet deduplication scans sorted packet IDs with two pointers. |

# 3.2 Sliding Window

## The Problem Situation

You need to find a subarray or substring of a specific size or satisfying a condition (max sum, longest without repeating, etc.). Brute force checks all $O(n^2)$ subarrays. Sliding window maintains a running window and expands or shrinks it in O(n) total.

## Key Insight

| | |
|---|---|
| □ **Insight** | Instead of recomputing from scratch for every subarray, maintain a window between two pointers. When the window becomes invalid (too large, has duplicates, etc.), shrink from the left. The total number of left and right moves is O(n). |

## Analogy

| | |
|---|---|
| □ **Mental Model** | A magnifying glass sliding across a page. You see a fixed-width section at a time. To move right, you add the new right character and drop the old left character. You never restart from the beginning — you just slide. |

## Basic Code Template

```
# Fixed-size window — max sum of subarray of size k
def max_sum_fixed(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]  # add right, remove left
        best = max(best, window_sum)
    return best
```

| | |
|---|---|
| ✓**This Template Solves** | LeetCode 643 — Maximum Average Subarray I • LeetCode 1456 — Maximum Number of Vowels in Substring |

## Alternative Template

```
# Variable-size window — longest substring without repeating chars
def length_of_longest_substring(s):
    seen  = {}   # char → last seen index
    left  = 0
    best  = 0
    for right, ch in enumerate(s):
        if ch in seen and seen[ch] >= left:
            left = seen[ch] + 1   # shrink window
        seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

| | |
|---|---|
| **✅This Template Solves** | LeetCode 3 — Longest Substring Without Repeating • LeetCode 209 — Minimum Size Subarray Sum • LeetCode 567 — Permutation in String |

## Example Walkthrough

```
# Example: LeetCode 3
# s = 'abcabcbb'
# right=0 (a): seen={a:0}, window='a',      len=1
# right=1 (b): seen={a:0,b:1}, window='ab',  len=2
# right=2 (c): seen={..c:2}, window='abc',   len=3
# right=3 (a): a seen at 0 >= left(0) → left=1
#              seen={a:3}, window='bca',      len=3
# right=4 (b): b seen at 1 >= left(1) → left=2
#              window='cab', len=3
# Answer: 3
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Fixed window | O(n) | O(k) or O(alphabet) |
| Variable window | O(n) | O(k) or O(alphabet) |

## Pattern Recognition Clues

- Keywords: subarray, substring, contiguous, window, at most k, longest/shortest
- Problem involves a range of consecutive elements
- You need max/min/count of something within a range that changes
- Brute force is $O(n^2)$ — sliding window reduces to O(n)

## Interview Traps

| | |
|---|---|
| □ **Trap** | Resetting the entire window on invalidity instead of just shrinking from the left |
| □ **Trap** | Off-by-one in window size: window length = right - left + 1, not right - left |
| □ **Trap** | Forgetting to update seen index — must update even when character is already in window |

### Edge Cases

- Empty string or array
- k larger than the array length
- All characters the same
- Window that requires counting (use Counter or defaultdict)

### Important Practice Problems

- LeetCode 3 — Longest Substring Without Repeating Characters
- LeetCode 76 — Minimum Window Substring
- LeetCode 209 — Minimum Size Subarray Sum
- LeetCode 567 — Permutation in String
- LeetCode 239 — Sliding Window Maximum

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Real-time streaming analytics systems compute moving averages using sliding windows. Network rate limiters track requests in a rolling time window. Log analysis systems find peak traffic windows. Video processing computes frame-level statistics over sliding temporal windows. |

## 3.3 Prefix Sum

### The Problem Situation

You need to answer multiple range sum queries on an array, or find a subarray whose sum equals a target. Naively computing each range sum is O(n) per query. Prefix sums allow O(1) range queries after O(n) preprocessing. Combined with a hash map, prefix sums solve subarray sum problems in O(n).

## Key Insight

| | |
|---|---|
| **Insight** | Precompute a running total. The sum of any subarray [i, j] = prefix[j+1] - prefix[i]. You convert a range sum query from O(n) work into a single subtraction. For finding subarrays with a given sum, store prefix sums in a hash map and look up the complement. |

## Analogy

| | |
|---|---|
| **Mental Model** | A bank account running balance. Instead of adding up all transactions between two dates, you take the balance on the end date and subtract the balance on the start date. One subtraction gives you the total — no matter how many transactions are in between. |

## Basic Code Template

```
# 1D Prefix Sum — range query in O(1)
def build_prefix(nums):
    prefix = [0] * (len(nums) + 1)
    for i, num in enumerate(nums):
        prefix[i+1] = prefix[i] + num
    return prefix

# Sum of subarray [left, right] inclusive
def range_sum(prefix, left, right):
    return prefix[right+1] - prefix[left]

# Usage
nums   = [1, 2, 3, 4, 5]
prefix = build_prefix(nums)  # [0,1,3,6,10,15]
print(range_sum(prefix, 1, 3))  # sum of [2,3,4] = 9
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 303 — Range Sum Query • LeetCode 304 — Range Sum Query 2D • LeetCode 1480 — Running Sum of 1D Array |

## Alternative Template

```
# Prefix Sum + Hash Map — subarray sum equals k
def subarray_sum(nums, k):
    count   = 0
    current = 0
    seen    = {0: 1}   # prefix_sum → frequency
    for num in nums:
        current += num
        # if (current - k) was seen before, a valid subarray ends here
        count += seen.get(current - k, 0)
        seen[current] = seen.get(current, 0) + 1
    return count
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 560 — Subarray Sum Equals K • LeetCode 974 — Subarray Sums Divisible by K • LeetCode 525 — Contiguous Array |

## Example Walkthrough

```
# Example: nums=[1,2,3], k=3
# current=0, seen={0:1}
# num=1: current=1, need=1-3=-2, seen.get(-2)=0, seen={0:1,1:1}
# num=2: current=3, need=3-3=0,  seen.get(0)=1  → count=1
#        seen={0:1,1:1,3:1}
# num=3: current=6, need=6-3=3,  seen.get(3)=1  → count=2
# Answer: 2  (subarrays [1,2] and [3])
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Build prefix array | O(n) | O(n) |
| Range query | O(1) | O(1) |
| Subarray sum with hashmap | O(n) | O(n) |

## Pattern Recognition Clues

- Keywords: range sum, subarray sum, sum equals k, cumulative, running total
- Multiple queries on the same array — build prefix once, answer in O(1)
- Find subarray with exact sum — prefix sum + hash map
- 2D grid range sum — 2D prefix sum

## Interview Traps

| | |
|---|---|
| ⬜ **Trap** | Off-by-one in indexing: prefix[i+1] = prefix[i] + nums[i] — the prefix array is size n+1 |
| ⬜ **Trap** | Forgetting to initialise seen = {0: 1} — misses subarrays that start from index 0 |
| ⬜ **Trap** | Using prefix sum on a problem that needs min/max — prefix sum only works for sum |

## Edge Cases

- Empty array

- All negative numbers
- k = 0 — subarrays that sum to zero
- Multiple subarrays with same prefix sum — count correctly with frequency map

### Important Practice Problems

- LeetCode 303 — Range Sum Query - Immutable
- LeetCode 560 — Subarray Sum Equals K
- LeetCode 974 — Subarray Sums Divisible by K
- LeetCode 525 — Contiguous Array
- LeetCode 304 — Range Sum Query 2D - Immutable

### Real World Python Usage

| | |
|---|---|
| **Real World** | Database GROUP BY with cumulative SUM uses prefix sum logic. Financial systems compute running P&L using prefix sums. Image processing uses 2D prefix sums (integral images) for fast region-average calculations. Pandas .cumsum() is a direct prefix sum implementation. |

## 3.4 Hash Map Counting

### The Problem Situation

You need to count frequencies, find duplicates, detect anagrams, or find a value that satisfies a relationship with another value seen earlier (like Two Sum). Hash maps give O(1) lookup and let you trade space for time — turning $O(n^2)$ brute force into O(n).

### Key Insight

| | |
|---|---|
| **Insight** | Store what you have seen so far in a hash map. For each new element, check if the map already contains what you need. This is the classic 'store and look up' pattern — you answer the question 'have I seen the complement of this before?' in O(1). |

### Analogy

| | |
|---|---|
| **Mental Model** | You are at a party looking for someone wearing the same colour shirt as you. Instead of asking every person ($O(n^2)$), you keep a list of shirt colours you have seen (O(n) space). For each new person you meet, you check your list in one second. |

## Basic Code Template

```
# Two Sum — classic hash map pattern
def two_sum(nums, target):
    seen = {}   # value → index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 1 — Two Sum • LeetCode 167 — Two Sum II • LeetCode 653 — Two Sum IV BST |

## Alternative Template

```
# Frequency map — group anagrams
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for word in words:
        key = tuple(sorted(word))   # anagrams share same sorted key
        groups[key].append(word)
    return list(groups.values())

# Top K frequent elements
from collections import Counter
def top_k_frequent(nums, k):
    freq = Counter(nums)
    return [x for x, _ in freq.most_common(k)]
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 49 — Group Anagrams • LeetCode 347 — Top K Frequent Elements • LeetCode 242 — Valid Anagram |

## Example Walkthrough

```
# Example: Two Sum — nums=[2,7,11,15], target=9
# i=0 (2):  complement=7,  not in seen → seen={2:0}
# i=1 (7):  complement=2,  2 in seen at index 0 → return [0,1]

# Example: Group Anagrams — ['eat','tea','tan','ate','nat','bat']
# 'eat' → sorted → ('a','e','t') → groups[('a','e','t')]=['eat']
# 'tea' → sorted → ('a','e','t') → groups[('a','e','t')]=['eat','tea']
# 'tan' → sorted → ('a','n','t') → groups[('a','n','t')]=['tan']
# ...
# Result: [['eat','tea','ate'], ['tan','nat'], ['bat']]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Single pass | O(n) | O(n) |
| Frequency count | O(n) | O(k) unique elements |

## Pattern Recognition Clues

- Keywords: two sum, frequency, count, duplicate, anagram, most common
- You need to find a complement or pair — store and look up
- You need to group items by a property — use hash map with list values
- Any $O(n^2)$ problem involving pair comparisons — try hash map for O(n)

## Interview Traps

| | |
|---|---|
| □ **Trap** | Returning the wrong type — Two Sum wants indices, not values |
| □ **Trap** | Using a list instead of a set/dict for membership check — O(n) vs O(1) |
| □ **Trap** | Forgetting to handle the case where the same element is used twice |

## Edge Cases

- nums contains duplicates — handle with index tracking
- Multiple valid answers — return first found or all
- Target is 0 and array has zeros

## Important Practice Problems

- LeetCode 1 — Two Sum
- LeetCode 49 — Group Anagrams
- LeetCode 347 — Top K Frequent Elements
- LeetCode 242 — Valid Anagram
- LeetCode 128 — Longest Consecutive Sequence

## Real World Python Usage

| | |
|---|---|
| □ **Real World** | Hash map counting powers inverted indexes in search engines (word → document list). Recommendation systems count user-item interactions in a hash map. Log parsers group events by type using frequency maps. Python's Counter is directly used in NLP for word frequency analysis. |

# 3.5 Binary Search

## The Problem Situation

You need to search in a sorted array, find the boundary of a condition, or determine the minimum/maximum value that satisfies a constraint. Binary search eliminates half the search space each step, giving O(log n) — versus O(n) linear scan.

## Key Insight

| | |
|---|---|
| □ **Insight** | At each step, check the middle element. If it matches your condition, record it as a candidate. Then decide whether the answer could be on the left half or right half and eliminate the other half. Repeat until the search space is empty. |

## Analogy

| | |
|---|---|
| □ **Mental Model** | Looking up a word in a physical dictionary. You open to the middle. If your word comes before the middle word, throw away the right half. If it comes after, throw away the left half. Repeat. You find any word in 20 steps in a 1,000,000-word dictionary. |

## Basic Code Template

```
# Classic binary search — find exact target
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2   # avoids overflow
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left  = mid + 1
        else:
            right = mid - 1
    return -1
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 704 — Binary Search • LeetCode 33 — Search in Rotated Sorted Array |

## Alternative Template

```
# Left boundary — first position satisfying condition
def left_bound(nums, target):
    left, right = 0, len(nums)
    while left < right:
        mid = (left + right) // 2
        if nums[mid] < target:  left  = mid + 1
        else:                   right = mid
    return left  # first index where nums[i] >= target

# Search on answer space — minimum max subarray sum
def can_split(nums, k, mid):
    count, total = 1, 0
    for num in nums:
        if total + num > mid: count += 1; total = 0
        total += num
    return count <= k

def split_array(nums, k):
    left, right = max(nums), sum(nums)
    while left < right:
        mid = (left + right) // 2
        if can_split(nums, k, mid): right = mid
        else:                       left  = mid + 1
    return left
```

| | |
|---|---|
| **✅This Template Solves** | LeetCode 278 — First Bad Version • LeetCode 410 — Split Array Largest Sum • LeetCode 875 — Koko Eating Bananas |

## Example Walkthrough

```
# Example: Search in Rotated Array — nums=[4,5,6,7,0,1,2], target=0
# left=0, right=6, mid=3 (val=7)
# Left half [4,5,6,7] is sorted: target(0) not in [4,7] → go right
# left=4, right=6, mid=5 (val=1)
# Right half [1,2] is sorted: target(0) not in [1,2] → go left
# left=4, right=4, mid=4 (val=0) → found at index 4
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Classic search | O(log n) | O(1) |
| Search on answer space | O(n log(max-min)) | O(1) |

## Pattern Recognition Clues

- Keywords: sorted, find target, first/last occurrence, minimum/maximum satisfying condition
- If you can define a monotonic condition (true/false boundary), binary search works
- Search space is a range of numbers (not just array indices) — binary search on answer
- Any time you see O(n log n) hinted — a binary search inside a loop

## Interview Traps

| | |
|---|---|
| **Trap** | Using left <= right for boundary search — causes infinite loop. Use left < right for boundaries. |
| **Trap** | Calculating mid as (left+right)//2 — can overflow in other languages; use left+(right-left)//2 |
| **Trap** | Forgetting to narrow the search space — not updating left or right leads to infinite loop |

## Edge Cases

- Single element array
- Target not present — return -1 or the insertion point
- Duplicates in array — affects leftmost/rightmost boundary
- Rotated sorted array — check which half is sorted first

## Important Practice Problems

- LeetCode 704 — Binary Search
- LeetCode 33 — Search in Rotated Sorted Array
- LeetCode 153 — Find Minimum in Rotated Sorted Array
- LeetCode 875 — Koko Eating Bananas
- LeetCode 410 — Split Array Largest Sum

## Real World Python Usage

| | |
|---|---|
| **Real World** | Python's bisect module uses binary search internally. Database B-tree indexes use binary search at each node. Git bisect finds the first bad commit using binary search. NumPy's searchsorted() is binary search on sorted arrays. CDN load balancers use binary search to select servers by sorted capacity. |

# 3.6 Monotonic Stack

## The Problem Situation

You need to find the next greater element, next smaller element, previous greater element, or the largest rectangle/area for each element. Brute force compares each element with every other — $O(n^2)$. A monotonic stack processes each element at most twice (once pushed, once popped) giving O(n).

## Key Insight

| | |
|---|---|
| **Insight** | Maintain a stack that is always sorted (increasing or decreasing). When a new element breaks the sorted order, pop elements from the stack — each popped element has found its answer (the current element is its next greater/smaller). The key insight: each element is pushed once and popped at most once, so the total work is O(n). |

## Analogy

| | |
|---|---|
| **Mental Model** | A line of people waiting to see who is taller than them. Each person looks back. If they are taller than the person behind them, that person behind has found their 'next taller person' (the current one) and steps out of line. The line always stays in height order. |

## Basic Code Template

```
# Next Greater Element — monotonic decreasing stack
def next_greater(nums):
    n      = len(nums)
    result = [-1] * n
    stack  = []   # stores indices
    for i in range(n):
        # while current element breaks the decreasing order
        while stack and nums[stack[-1]] < nums[i]:
            idx = stack.pop()
            result[idx] = nums[i]   # nums[i] is the next greater
        stack.append(i)
    return result
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 496 — Next Greater Element I • LeetCode 739 — Daily Temperatures • LeetCode 503 — Next Greater Element II |

## Alternative Template

```
# Largest Rectangle in Histogram — monotonic increasing stack
def largest_rectangle(heights):
```

```
    stack    = []   # increasing stack of indices
    max_area = 0
    heights  = heights + [0]   # sentinel to flush stack at end
    for i, h in enumerate(heights):
        start = i
        while stack and heights[stack[-1]] > h:
            idx      = stack.pop()
            width    = i - (stack[-1] if stack else -1) - 1
            max area = max(max area, heights[idx] * width)
            start    = idx
        stack.append(start)
    return max_area
```

| ✅This Template Solves | LeetCode 84 — Largest Rectangle in Histogram • LeetCode 85 — Maximal Rectangle |
|---|---|

## Example Walkthrough

```
# Example: Daily Temperatures — [73,74,75,71,69,72,76,73]
# i=0 (73): stack=[]   → push 0.     stack=[0]
# i=1 (74): 74>73 → pop 0, result[0]=1. push 1. stack=[1]
# i=2 (75): 75>74 → pop 1, result[1]=1. push 2. stack=[2]
# i=3 (71): 71<75 → push 3.  stack=[2,3]
# i=4 (69): 69<71 → push 4.  stack=[2,3,4]
# i=5 (72): 72>69 → pop 4, result[4]=1
#            72>71 → pop 3, result[3]=2. push 5. stack=[2,5]
# i=6 (76): 76>72 → pop 5, result[5]=1
#            76>75 → pop 2, result[2]=4. push 6. stack=[6]
# Result: [1,1,4,2,1,1,0,0]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| All patterns | O(n) | O(n) |
| Each element pushed and popped once | | |

## Pattern Recognition Clues

- Keywords: next greater, next smaller, previous greater, previous smaller, span, histogram
- You need to find the nearest element that is larger or smaller
- Area or span problems — largest rectangle, maximal rectangle
- Problem where each element 'waits' for something — use a stack to store waiting elements

### Interview Traps

| | |
|---|---|
| □ **Trap** | Storing values in stack instead of indices — you often need the index to compute distances/widths |
| □ **Trap** | Choosing wrong monotonic direction — decreasing stack for next greater, increasing for next smaller |
| □ **Trap** | Forgetting to handle remaining elements in stack after loop ends |

### Edge Cases

- Array with all elements equal
- Strictly increasing or decreasing array
- Single element array
- Circular array — process array twice (LeetCode 503)

### Important Practice Problems

- LeetCode 739 — Daily Temperatures
- LeetCode 496 — Next Greater Element I
- LeetCode 84 — Largest Rectangle in Histogram
- LeetCode 85 — Maximal Rectangle
- LeetCode 42 — Trapping Rain Water

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Stock span calculations in financial systems use monotonic stacks. Compilers use monotonic stack-like structures for operator precedence parsing. Image processing computes local maxima using a monotonic approach. Python AST evaluation uses stack-based expression parsing. |

## 3.7 Monotonic Queue

---

### The Problem Situation

You need the maximum or minimum in a sliding window of size k as the window moves across the array. Naively recalculating the max for each window is O(nk). A monotonic deque maintains the maximum at the front in O(1), giving O(n) total.

## Key Insight

| | |
|---|---|
| **Insight** | Maintain a deque of indices in decreasing order of their values. When the window slides right, remove the front if it is outside the window. Remove all elements from the back that are smaller than the new element (they can never be the max while the new element is in the window). The front of the deque is always the maximum of the current window. |

## Analogy

| | |
|---|---|
| **Mental Model** | A queue of people ranked by height. When a taller person joins at the back, all shorter people already in the queue behind them are useless — they will never be the tallest while the taller person is present. So shorter people leave when a taller person arrives. |

## Basic Code Template

```
# Sliding window maximum — monotonic decreasing deque
from collections import deque

def max_sliding_window(nums, k):
    dq     = deque()   # stores indices, values decreasing
    result = []
    for i, num in enumerate(nums):
        # remove indices outside the window
        while dq and dq[0] < i - k + 1:
            dq.popleft()
        # remove smaller elements from back — they are useless
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        # window is fully formed
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 239 — Sliding Window Maximum • LeetCode 1438 — Longest Subarray With Absolute Diff <= Limit |

## Example Walkthrough

```
# Example: nums=[1,3,-1,-3,5,3,6,7], k=3
# i=0 (1):  dq=[0]
# i=1 (3):  3>1 → pop 0, dq=[1]
# i=2 (-1): -1<3 → dq=[1,2],  window full → result=[nums[1]]=[ 3]
```

```
# i=3 (-3): -3<-1 → dq=[1,2,3], result=[3, 3]
# i=4 (5):  5>-3→pop3, 5>-1→pop2, 5>3→pop1, dq=[4], result=[3,3,5]
# i=5 (3):  3<5 → dq=[4,5], result=[3,3,5,5]
# i=6 (6):  6>3→pop5, 6>5→pop4, dq=[6], result=[3,3,5,5,6]
# i=7 (7):  7>6→pop6, dq=[7], result=[3,3,5,5,6,7]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| All cases | O(n) | O(k) |
| Each element enqueued and dequeued once | | |

## Pattern Recognition Clues

- Keywords: sliding window maximum, sliding window minimum, window of size k
- You need max/min in every window of fixed size as it slides — monotonic queue gives O(n)
- Often combined with other patterns — sliding window + monotonic queue

## Interview Traps

| | |
|---|---|
| □ **Trap** | Storing values instead of indices — you need indices to check if elements are outside the window |
| □ **Trap** | Wrong removal condition: remove front when dq[0] < i - k + 1 (strictly less) |
| □ **Trap** | Confusing with monotonic stack — queue removes from both ends, stack only from top |

## Edge Cases

- k = 1 — every element is its own window maximum
- k = len(nums) — answer is the global maximum
- All elements equal

## Important Practice Problems

- LeetCode 239 — Sliding Window Maximum
- LeetCode 1438 — Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit

- LeetCode 862 — Shortest Subarray with Sum at Least K

### Real World Python Usage

| | |
|---|---|
| **Real World** | Real-time sensor data monitoring uses sliding window max to detect anomalies. Video encoding tracks maximum frame complexity in a window. Trading systems compute rolling max/min for technical indicators. Python pandas .rolling().max() uses an efficient sliding window internally. |

## 3.8 Fast and Slow Pointers (Floyd's Algorithm)

### The Problem Situation

You are working with a linked list and need to detect a cycle, find the middle node, or find the start of a cycle. Two pointers moving at different speeds will eventually meet if a cycle exists — or the fast pointer will reach the end if no cycle exists.

### Key Insight

| | |
|---|---|
| **Insight** | Move the slow pointer one step at a time and the fast pointer two steps at a time. If there is a cycle, fast will lap slow and they will meet. If there is no cycle, fast reaches None first. To find the middle: when fast reaches the end, slow is at the middle. |

### Analogy

| | |
|---|---|
| **Mental Model** | Two runners on a circular track. One runs at double the speed. If the track is circular (has a cycle), the faster runner will eventually lap the slower runner — they will meet. If the track has an endpoint, the faster runner just finishes first. |

### Basic Code Template

```
# Cycle detection — Floyd's Tortoise and Hare
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

| ✅This Template Solves | LeetCode 141 — Linked List Cycle • LeetCode 142 — Linked List Cycle II |
|---|---|

## Alternative Template

```
# Find middle of linked list
def find_middle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow   # slow is at the middle

# Find start of cycle
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            # reset slow to head, keep fast at meeting point
            slow = head
            while slow != fast:
                slow = slow.next
                fast = fast.next
            return slow   # start of cycle
    return None
```

| ✅This Template Solves | LeetCode 142 — Linked List Cycle II • LeetCode 876 — Middle of the Linked List • LeetCode 234 — Palindrome Linked List |
|---|---|

## Example Walkthrough

```
# Example: has_cycle — 1→2→3→4→2 (cycle at node 2)
# Step 1: slow=2, fast=3
# Step 2: slow=3, fast=2  (fast wraps through cycle)
# Step 3: slow=4, fast=4  → MEET → cycle detected

# Example: find_middle — 1→2→3→4→5
# Start: slow=1, fast=1
# Step 1: slow=2, fast=3
# Step 2: slow=3, fast=5
# fast.next is None → stop. slow=3 is the middle
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|

| | | |
|---|---|---|
| Cycle detection | O(n) | O(1) |
| Find middle | O(n) | O(1) |
| Find cycle start | O(n) | O(1) |

## Pattern Recognition Clues

- Keywords: linked list cycle, middle of list, palindrome linked list, happy number
- Problem is on a linked list and requires O(1) space — fast/slow pointers
- Any problem where two things move at different rates in a sequence
- Happy number problem — same pattern applied to number sequences

## Interview Traps

| | |
|---|---|
| **Trap** | Checking slow == fast before the first move — they start at the same node |
| **Trap** | Not checking fast and fast.next before advancing — causes NoneType AttributeError |
| **Trap** | Returning slow directly for even-length list middle — returns second of two middle nodes |

## Edge Cases

- Empty list or single node
- List with exactly two nodes
- Cycle starting at head
- Even vs odd length list for middle finding

## Important Practice Problems

- LeetCode 141 — Linked List Cycle
- LeetCode 142 — Linked List Cycle II
- LeetCode 876 — Middle of the Linked List
- LeetCode 234 — Palindrome Linked List
- LeetCode 202 — Happy Number

## Real World Python Usage

| | |
|---|---|
| **Real World** | Operating system deadlock detection algorithms use cycle detection in resource allocation graphs. Memory leak detectors use pointer-chasing to find circular |

| | |
|---|---|
| | references. Rho algorithm (Pollard's) for integer factorisation uses the fast/slow pointer concept. Python's gc module detects reference cycles using graph-based cycle detection. |

## 3.9 Depth-First Search (DFS)

### The Problem Situation

You need to explore all paths in a tree or graph, check connectivity, find a path, or compute a property that depends on subtree results. DFS goes as deep as possible down one path before backtracking. It is the foundation of tree traversal, graph exploration, and backtracking.

### Key Insight

| | |
|---|---|
| **Insight** | Use recursion (which uses the call stack) or an explicit stack. Visit a node, mark it, then recurse into each unvisited neighbour. The recursive structure naturally handles the backtracking — when a recursive call returns, you are automatically back at the parent node. |

### Analogy

| | |
|---|---|
| **Mental Model** | Exploring a maze by always turning left. You go as deep as possible down one corridor. When you hit a dead end, you backtrack to the last junction and try the next corridor. You always explore one complete path before trying alternatives. |

### Basic Code Template

```
# DFS on a tree — recursive
def dfs_tree(root):
    if not root:
        return
    # Pre-order: process node BEFORE children
    print(root.val)
    dfs_tree(root.left)
    dfs_tree(root.right)
    # Post-order: process node AFTER children
    # (move print to after the recursive calls)
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 104 — Maximum Depth of Binary Tree • LeetCode 112 — Path Sum • LeetCode 543 — Diameter of Binary Tree |

## Alternative Template

```
# DFS on a graph — with visited set
from collections import defaultdict

def dfs_graph(graph, node, visited):
    visited.add(node)
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs_graph(graph, neighbour, visited)

# DFS iterative — explicit stack (avoids recursion limit)
def dfs_iterative(graph, start):
    visited = set()
    stack   = [start]
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        for nb in graph[node]:
            if nb not in visited:
                stack.append(nb)
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 200 — Number of Islands • LeetCode 133 — Clone Graph • LeetCode 695 — Max Area of Island |

## Example Walkthrough

```
# Example: Max Depth — tree: 3→[9, 20→[15,7]]
# dfs(3): 1 + max(dfs(9), dfs(20))
# dfs(9): 1 + max(dfs(None),dfs(None)) = 1+0 = 1
# dfs(20): 1 + max(dfs(15),dfs(7)) = 1+max(1,1) = 2
# dfs(3): 1 + max(1, 2) = 3
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Tree DFS | O(n) | O(h) |
| Graph DFS | O(V+E) | O(V) |
| h=tree height, V=vertices, E=edges | | |

## Pattern Recognition Clues

- Keywords: all paths, connected components, explore, traverse, depth, subtree

- Tree problems almost always use DFS — traverse and compute something on each subtree
- Graph problems: count components, check connectivity, detect cycles
- When you need to process children before parent — use post-order DFS

### Interview Traps

| | |
|---|---|
| □ **Trap** | Forgetting the visited set on graphs — causes infinite loop on cycles |
| □ **Trap** | Using DFS instead of BFS for shortest path — DFS does not guarantee shortest path |
| □ **Trap** | Hitting Python recursion limit on deep trees — use iterative DFS with explicit stack |

### Edge Cases

- Empty graph or tree
- Disconnected graph — need outer loop to start DFS from every unvisited node
- Graph with self-loops
- Very deep recursion — switch to iterative

### Important Practice Problems

- LeetCode 104 — Maximum Depth of Binary Tree
- LeetCode 200 — Number of Islands
- LeetCode 133 — Clone Graph
- LeetCode 543 — Diameter of Binary Tree
- LeetCode 695 — Max Area of Island

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Web crawlers use DFS to explore website link structures. Python's import system resolves module dependencies using DFS. Git's commit graph traversal uses DFS. File system traversal (os.walk) uses DFS. Compilers use DFS on the AST during code generation. |

## 3.10 Breadth-First Search (BFS)

## The Problem Situation

You need to find the shortest path in an unweighted graph, traverse a tree level by level, or find minimum steps to reach a state. BFS explores all nodes at distance 1 before distance 2, distance 2 before distance 3 — guaranteeing the shortest path in unweighted graphs.

## Key Insight

| | |
|---|---|
| **Insight** | Use a queue. Start by enqueuing the source. At each step, dequeue a node, process it, and enqueue all unvisited neighbours. Because a queue is FIFO, nodes are processed in order of their distance from the source — the first time you reach a node is via the shortest path. |

## Analogy

| | |
|---|---|
| **Mental Model** | Dropping a stone in a pond. Ripples spread outward in circles — level 1 is all nodes one step from source, level 2 is two steps, and so on. BFS visits nodes exactly in this ripple order. |

## Basic Code Template

```
# BFS — shortest path in unweighted graph
from collections import deque

def bfs(graph, start, target):
    visited = set([start])
    queue   = deque([(start, 0)])   # (node, distance)
    while queue:
        node, dist = queue.popleft()
        if node == target:
            return dist
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, dist + 1))
    return -1   # not reachable
```

| | |
|---|---|
| **This Template Solves** | LeetCode 102 — Binary Tree Level Order • LeetCode 994 — Rotting Oranges • LeetCode 127 — Word Ladder |

## Alternative Template

```
# Multi-source BFS — multiple starting nodes simultaneously
def multi_source_bfs(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    # enqueue all sources at the start
```

```
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:   # source condition
                queue.append((r, c, 0))
    visited = set((r,c) for r,c,_ in queue)
    dirs = [(0,1),(0,-1),(1,0),(-1,0)]
    while queue:
        r, c, dist = queue.popleft()
        for dr, dc in dirs:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited:
                visited.add((nr,nc))
                queue.append((nr, nc, dist+1))
```

| | |
|---|---|
| **✅This Template Solves** | LeetCode 994 — Rotting Oranges • LeetCode 286 — Walls and Gates • LeetCode 542 — 01 Matrix |

## Example Walkthrough

```
# Example: BFS level-order tree — [3,9,20,null,null,15,7]
# Queue: [3]
# Dequeue 3 → enqueue 9,20. Level=[3]
# Queue: [9,20]
# Dequeue 9 → no children. Dequeue 20 → enqueue 15,7. Level=[9,20]
# Queue: [15,7]
# Dequeue 15,7 → no children. Level=[15,7]
# Result: [[3],[9,20],[15,7]]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| BFS traversal | O(V+E) | O(V) |
| Level order tree | O(n) | O(w) max width |
| Multi-source BFS | O(V+E) | O(V) |

## Pattern Recognition Clues

- Keywords: shortest path, minimum steps, level order, nearest, fewest moves
- Unweighted graph and you need shortest path — always BFS, never DFS
- Tree level-by-level processing — BFS with level tracking
- Multiple sources spreading simultaneously — multi-source BFS

## Interview Traps

| | |
|---|---|
| □ **Trap** | Using a list instead of deque for the queue — list.pop(0) is O(n), deque.popleft() is O(1) |
| □ **Trap** | Adding nodes to visited when dequeuing instead of enqueuing — causes duplicate processing |
| □ **Trap** | Forgetting to mark visited when enqueuing — same node can be added to queue multiple times |

### Edge Cases

- Disconnected graph — BFS from one node does not reach all nodes
- Target not reachable — return -1 or infinity
- Grid BFS — check bounds before enqueuing
- Level tracking — use len(queue) at start of each level loop

### Important Practice Problems

- LeetCode 102 — Binary Tree Level Order Traversal
- LeetCode 994 — Rotting Oranges
- LeetCode 127 — Word Ladder
- LeetCode 542 — 01 Matrix
- LeetCode 286 — Walls and Gates

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Google Maps uses BFS for finding nearby places. Social networks use BFS for friend-of-friend suggestions. Web crawlers use BFS for breadth-first URL discovery. Netflix recommendation engine uses BFS-like expansion on its content graph. Python's importlib uses BFS-like resolution for circular import detection. |

# Pattern Comparison — Section 3A

| Pattern | Best For | Core Idea | Key Data Structure |
|---|---|---|---|
| Two Pointers | Sorted arrays, pairs, partitioning | Move left/right toward each other | Array indices |

| | | | |
|---|---|---|---|
| Sliding Window | Contiguous subarray/substring | Expand right, shrink left | Hash map / Counter |
| Prefix Sum | Range sum, subarray sum = k | Precompute running total | Array + Hash map |
| Hash Map Count | Frequency, pairs, grouping | Store and look up complement | dict / Counter |
| Binary Search | Sorted array, answer space | Eliminate half each step | Array indices |
| Monotonic Stack | Next greater/smaller, histogram | Stack stays sorted, pop when broken | Stack (list) |
| Monotonic Queue | Sliding window max/min | Deque stays sorted, pop stale elements | deque |
| Fast/Slow Ptr | Cycle detection, middle of list | Two speeds, meet if cycle | Linked list pointers |
| DFS | All paths, subtree computation | Go deep, backtrack | Recursion / Stack |
| BFS | Shortest path, level order | Process level by level | deque (queue) |

---

**End of Section 3A.** Continue with Section 3B — Patterns 3.11 to 3.20.

# Python & Algorithms for Coding Interviews

Section 3B — Core Algorithmic Patterns (3.11 – 3.20)

## 3.11 Backtracking

### The Problem Situation

You need to generate all possible combinations, subsets, permutations, or find all paths satisfying a constraint. Backtracking explores every possibility but prunes branches that cannot lead to a valid answer, making it far more efficient than brute force enumeration.

### Key Insight

| | |
|---|---|
| **Insight** | Build the solution incrementally. At each step, make a choice and recurse. If the current path is invalid or complete, undo the choice (backtrack) and try the next option. The key operation is: choose → explore → unchoose. |

### Analogy

| | |
|---|---|
| **Mental Model** | Solving a Sudoku puzzle. You write a number in a cell, then try to fill the rest. If you get stuck (a cell has no valid number), you erase your last choice and try the next number. You explore all possibilities but stop exploring any path the moment it becomes invalid. |

### Basic Code Template

```
# Universal backtracking template
def backtrack(state, choices, result):
    if is_complete(state):      # base case
        result.append(list(state))
        return
    for choice in choices:
        if is valid(state, choice):  # prune invalid paths
            state.append(choice)     # choose
            backtrack(state, choices, result)  # explore
```

```
            state.pop()              # unchoose (backtrack)

# Subsets — include or exclude each element
def subsets(nums):
    result = []
    def bt(start, path):
        result.append(list(path))
        for i in range(start, len(nums)):
            path.append(nums[i])
            bt(i + 1, path)
            path.pop()
    bt(0, [])
    return result
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 78 — Subsets • LeetCode 90 — Subsets II • LeetCode 77 — Combinations |

## Alternative Template

```
# Permutations
def permutations(nums):
    result = []
    def bt(path, remaining):
        if not remaining:
            result.append(list(path))
            return
        for i in range(len(remaining)):
            path.append(remaining[i])
            bt(path, remaining[:i] + remaining[i+1:])
            path.pop()
    bt([], nums)
    return result

# Combination Sum — reuse elements, sum to target
def combination sum(candidates, target):
    result = []
    def bt(start, path, remaining):
        if remaining == 0:
            result.append(list(path)); return
        if remaining < 0: return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            bt(i, path, remaining - candidates[i])  # i not i+1 (reuse
allowed)
            path.pop()
    bt(0, [], target)
    return result
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 46 — Permutations • LeetCode 39 — Combination Sum • LeetCode 131 — Palindrome Partitioning |

## Example Walkthrough

```
# Example: subsets([1,2,3])
# bt(0,[]): append [] → try 1
#   bt(1,[1]): append [1] → try 2
#     bt(2,[1,2]): append [1,2] → try 3
#       bt(3,[1,2,3]): append [1,2,3]
#     pop 3 → pop 2 → try 3
#     bt(3,[1,3]): append [1,3]
#   pop → try 2,3 similarly
# Result: [[],[1],[1,2],[1,2,3],[1,3],[2],[2,3],[3]]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Subsets | O(2^n) | O(n) |
| Permutations | O(n!) | O(n) |
| Combinations | O(C(n,k)) | O(k) |

## Pattern Recognition Clues

- Keywords: all combinations, all subsets, all permutations, generate all, find all paths
- You must enumerate all valid configurations — backtracking is the default tool
- Decision tree — at each node you make a binary choice (include/exclude) or k-ary choice
- Constraint satisfaction — Sudoku, N-Queens, word search on grid

## Interview Traps

| | |
|---|---|
| □ **Trap** | Appending the path object directly (result.append(path)) — appends a reference, not a copy. Always append list(path). |
| □ **Trap** | Not incrementing start index — causes duplicates in combination problems |
| □ **Trap** | Missing the pruning condition — without pruning, backtracking is just brute force |

## Edge Cases

- Empty input — return [[]] for subsets, [] for permutations
- Duplicates in input — sort first and skip same-value siblings (LeetCode 40, 90)
- Target = 0 in combination sum — empty combination is valid

### Important Practice Problems

- LeetCode 78 — Subsets
- LeetCode 46 — Permutations
- LeetCode 39 — Combination Sum
- LeetCode 79 — Word Search
- LeetCode 131 — Palindrome Partitioning

### Real World Python Usage

| | |
|---|---|
| **Real World** | Backtracking powers constraint solvers in AI planning systems. SAT solvers (used in chip design verification) use DPLL — a backtracking algorithm. Python's re module uses backtracking for regex matching. Puzzle solvers (Sudoku, crosswords) in apps use backtracking. |

## 3.12 Recursion and Divide & Conquer

### The Problem Situation

A problem can be broken into smaller subproblems of the same type. Solving each subproblem independently and combining results gives the final answer. Merge Sort, Quick Sort, and most tree problems use this pattern. Recursion is the mechanism; Divide & Conquer is the strategy.

### Key Insight

| | |
|---|---|
| **Insight** | Divide the problem into smaller subproblems. Conquer each subproblem recursively. Combine the results. The base case is the smallest problem you can solve directly. The recursive case reduces the problem size each call, guaranteeing termination. |

### Analogy

| | |
|---|---|
| **Mental Model** | Breaking a large team project into subtasks. Each subtask is assigned to a smaller team (recursive call). When each subtask is done, the results are merged into the final deliverable. The smallest subtask (a single person's work) is the base case. |

### Basic Code Template

```
# Recursive function structure
def solve(problem):
    # Base case — smallest solvable problem
```

```
    if is_base_case(problem):
        return base_solution
    # Divide
    left_part, right_part = divide(problem)
    # Conquer
    left_result  = solve(left_part)
    right_result = solve(right_part)
    # Combine
    return combine(left result, right result)

# Merge Sort — classic divide and conquer
def merge sort(nums):
    if len(nums) <= 1:
        return nums
    mid   = len(nums) // 2
    left  = merge_sort(nums[:mid])
    right = merge_sort(nums[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j  = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]: result.append(left[i]);  i += 1
        else:                   result.append(right[j]); j += 1
    return result + left[i:] + right[j:]
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 912 — Sort an Array • LeetCode 23 — Merge K Sorted Lists • LeetCode 240 — Search a 2D Matrix II |

## Alternative Template

```
# Understanding the call stack during recursion
# fibonacci(4):
#   fibonacci(3)
#     fibonacci(2)
#       fibonacci(1) → 1
#       fibonacci(0) → 0
#     return 1
#     fibonacci(1) → 1
#   return 2
#   fibonacci(2)
#     fibonacci(1) → 1
#     fibonacci(0) → 0
#   return 1
# return 3

# Without memoization: O(2^n) — same subproblems computed repeatedly
# With @lru_cache:     O(n)   — each subproblem computed once
from functools import lru_cache
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 169 — Majority Element • LeetCode 53 — Maximum Subarray (D&C approach) • LeetCode 215 — Kth Largest Element |

## Example Walkthrough

```
# Example: Maximum Subarray via Divide & Conquer
# nums = [-2,1,-3,4,-1,2,1,-5,4]
# Divide at mid=4:
#   left  = [-2,1,-3,4,-1]  → max crossing includes index 4 (-1)
#   right = [2,1,-5,4]      → max crossing includes index 5 (2)
# Cross-max: scan left from mid → right from mid+1
# left_sum: -1,3,0,-2,-4  → max=3
# right_sum: 2,3,-2,2     → max=3
# cross = 3+3 = 6
# Answer: max(left_result, right_result, cross) = 6
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Merge Sort | O(n log n) | O(n) |
| Binary Search (D&C) | O(log n) | O(1) |
| Tree recursion (balanced) | O(n) | O(log n) |

## Pattern Recognition Clues

- Keywords: tree problems, sort, find Kth, majority, largest subarray — often D&C
- Problem structure naturally splits into independent halves
- When memoization would help — recursive tree with repeated subproblems → add @lru_cache
- Recursion on a tree is D&C — solve left subtree, solve right subtree, combine

## Interview Traps

| | |
|---|---|
| ⬜ **Trap** | Missing base case — infinite recursion and stack overflow |
| ⬜ **Trap** | Not reducing problem size in recursive call — call must get strictly smaller each time |
| ⬜ **Trap** | Forgetting @lru_cache on recursive solutions with overlapping subproblems — O(2^n) vs O(n) |

### Edge Cases

- n = 0 or 1 — base cases must handle all minimal inputs
- Very deep recursion on large n — switch to iterative or increase recursion limit
- Overlapping subproblems — add memoization to avoid recomputation

### Important Practice Problems

- LeetCode 912 — Sort an Array
- LeetCode 23 — Merge K Sorted Lists
- LeetCode 169 — Majority Element
- LeetCode 215 — Kth Largest Element in an Array
- LeetCode 53 — Maximum Subarray

### Real World Python Usage

| | |
|---|---|
| **Real World** | Python's built-in sort (Timsort) uses a hybrid of merge sort and insertion sort. NumPy's FFT uses divide and conquer on signal arrays. MapReduce (Hadoop, Spark) is an engineering-scale divide and conquer paradigm. Distributed systems split large datasets using recursive partitioning. |

## 3.13 Dynamic Programming (DP)

### The Problem Situation

A problem has two properties: overlapping subproblems (the same smaller problem is solved multiple times) and optimal substructure (the optimal solution to the whole problem contains optimal solutions to subproblems). DP solves each subproblem exactly once and stores the result — converting exponential time to polynomial time.

### Key Insight

| | |
|---|---|
| **Insight** | Identify what changes between subproblems — that is your state. Define dp[state] = answer for that state. Write a recurrence relation: dp[state] = f(dp[smaller_state]). Either use top-down memoization (@lru_cache) or build bottom-up from the smallest state to the largest. |

### Analogy

| | |
|---|---|
| **□ Mental Model** | Climbing stairs. You need to reach step n. You know you can reach step n from step n-1 or n-2. So you just need to know: how many ways to reach n-1 and n-2? You store those answers so you never compute them twice. Each step's answer builds on the previous steps' answers. |

## Basic Code Template

```
# Top-Down DP — Memoization with @lru_cache
from functools import lru_cache

# Fibonacci — classic DP
@lru_cache(maxsize=None)
def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)

# Coin Change — min coins to make amount
@lru_cache(maxsize=None)
def coin_change_memo(coins, amount):
    if amount == 0: return 0
    if amount  < 0: return float('inf')
    return 1 + min(coin_change_memo(coins, amount - c) for c in coins)
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 70 — Climbing Stairs • LeetCode 322 — Coin Change • LeetCode 1143 — LCS |

## Alternative Template

```
# Bottom-Up DP — Tabulation
# Coin Change iterative
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0   # base case: 0 coins to make amount 0
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                dp[a] = min(dp[a], 1 + dp[a - coin])
    return dp[amount] if dp[amount] != float('inf') else -1

# Longest Common Subsequence (LCS) — 2D DP
def lcs(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]
```

```
# 0/1 Knapsack
def knapsack(weights, values, capacity):
    n  = len(weights)
    dp = [[0]*(capacity+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(capacity+1):
            dp[i][w] = dp[i-1][w]  # don't take item i
            if weights[i-1] <= w:
                dp[i][w] = max(dp[i][w], values[i-1] + dp[i-1][w-weights[i-
1]])
    return dp[n][capacity]
```

| | |
|---|---|
| **✅This Template Solves** | LeetCode 322 — Coin Change • LeetCode 1143 — LCS • LeetCode 300 — LIS • LeetCode 416 — Partition Equal Subset Sum |

## Example Walkthrough

```
# Recurrence relations — the heart of DP
# Climbing Stairs:   dp[i] = dp[i-1] + dp[i-2]
# Coin Change:       dp[a] = min(dp[a-coin]+1) for coin in coins
# LCS:               dp[i][j] = dp[i-1][j-1]+1   if match
#                             = max(dp[i-1][j], dp[i][j-1]) if no match
# 0/1 Knapsack:      dp[i][w] = max(dp[i-1][w],
#                                   val[i]+dp[i-1][w-wt[i]])
# LIS:               dp[i] = max(dp[j]+1) for j<i if nums[j]<nums[i]

# Common DP subpatterns:
# Linear DP      → dp[i] depends on dp[i-1] or dp[i-k]
# 2D DP          → dp[i][j] depends on dp[i-1][j], dp[i][j-1]
# Interval DP    → dp[i][j] = best way to solve subarray [i..j]
# DP on strings  → LCS, edit distance, palindrome
# Knapsack       → include/exclude each item, capacity constraint
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Top-down memo | O(states) | O(states) |
| Bottom-up tab | O(states) | O(states) |
| Space optimised (rolling) | O(states) | O(1 row) |

## Pattern Recognition Clues

- Keywords: minimum/maximum, count ways, is it possible, optimal, partition
- Overlapping subproblems — same inputs computed multiple times in naive recursion

- Optimal substructure — optimal answer uses optimal answers to subproblems
- Choices at each step — include/exclude, take/skip, move direction

### Interview Traps

| | |
|---|---|
| □ **Trap** | Incorrect base cases — dp[0] and dp[1] must be set carefully before the loop |
| □ **Trap** | Wrong transition direction — ensure dp[i] only depends on already-computed states |
| □ **Trap** | 2D DP off-by-one — dp is size (m+1)×(n+1) but strings are indexed 0 to m-1 |

### Edge Cases

- Amount = 0 or empty input → should return 0 or empty
- No valid solution exists → return -1 or float('inf')
- Duplicate elements in knapsack inputs
- Very large state space — consider space optimisation with rolling array

### Important Practice Problems

- LeetCode 70 — Climbing Stairs
- LeetCode 322 — Coin Change
- LeetCode 300 — Longest Increasing Subsequence
- LeetCode 1143— Longest Common Subsequence
- LeetCode 416 — Partition Equal Subset Sum

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | DP powers autocorrect and spell check (edit distance). Sequence alignment in bioinformatics (DNA matching) uses LCS-style DP. Python's difflib module uses DP for text diff. Video codec compression (H.264) uses DP for motion estimation. Google's PageRank can be formulated as a DP on a graph. |

## 3.14 Greedy Algorithms

---

### The Problem Situation

At each step you make the locally optimal choice — the choice that looks best right now — without reconsidering previous choices. Greedy works when the local optimal choice at every step leads to the globally optimal solution. It does not work when local decisions have downstream side effects that need to be reconsidered.

## Key Insight

| | |
|---|---|
| **Insight** | Make the best possible decision at every step and never look back. Greedy works when the problem has the greedy choice property: choosing the locally best option at each step leads to the globally best outcome. When this property holds, greedy is far simpler and faster than DP. |

## Analogy

| | |
|---|---|
| **Mental Model** | Giving change with the fewest coins (using large coins first) — works with standard coin denominations. At each step, take the largest coin that fits. This works because standard coins (1,5,10,25 cents) have a special mathematical property. With arbitrary coin values, this greedy fails — you would need DP. |

## Basic Code Template

```
# Activity Selection — greedy by earliest end time
def activity_selection(intervals):
    intervals.sort(key=lambda x: x[1])  # sort by end time
    count, last_end = 1, intervals[0][1]
    for start, end in intervals[1:]:
        if start >= last_end:            # non-overlapping
            count   += 1
            last_end = end
    return count

# Jump Game — can you reach the end?
def can_jump(nums):
    max_reach = 0
    for i, jump in enumerate(nums):
        if i > max_reach: return False   # stuck
        max_reach = max(max_reach, i + jump)
    return True
```

| | |
|---|---|
| **This Template Solves** | LeetCode 55 — Jump Game • LeetCode 435 — Non-overlapping Intervals • LeetCode 45 — Jump Game II |

## Alternative Template

```
# Interval scheduling — merge or count overlapping intervals
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
```

```
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:              # overlapping
            merged[-1][1] = max(merged[-1][1], end)   # extend
        else:
            merged.append([start, end])
    return merged
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 56 — Merge Intervals • LeetCode 57 — Insert Interval • LeetCode 253 — Meeting Rooms II |

## Example Walkthrough

```
# Example: Non-overlapping Intervals
# intervals = [[1,2],[2,3],[3,4],[1,3]]
# Sort by end: [[1,2],[2,3],[1,3],[3,4]]
# Keep [1,2]: last_end=2
# [2,3]: start(2) >= last_end(2) → keep, last_end=3
# [1,3]: start(1) < last end(3)  → REMOVE (overlap)
# [3,4]: start(3) >= last_end(3) → keep
# Removed 1 interval → answer = 1
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Interval sorting | O(n log n) | O(1) or O(n) |
| Single pass greedy | O(n) | O(1) |

## Pattern Recognition Clues

- Keywords: minimum number of steps, maximum non-overlapping, earliest deadline, least cost
- Sorting by a greedy criterion (end time, value/weight ratio) often reveals the greedy order
- If DP feels like overkill and the problem has a natural ordering — try greedy first
- Interval problems almost always have a greedy solution

## Interview Traps

| | |
|---|---|
| ⚠ **Trap** | Applying greedy when it does not have the greedy choice property — leads to wrong answers. Always verify with a small counterexample. |
| ⚠ **Trap** | Wrong sorting key — greedy is very sensitive to sort order (end time vs start time matters) |

| | |
|---|---|
| □ **Trap** | Confusing greedy with DP — greedy never reconsiders; DP does. If you need to reconsider, use DP. |

### Edge Cases

- No intervals / empty array — return 0
- All intervals overlap — must remove all but one
- Negative jumps in jump game — stuck from the start

### Important Practice Problems

- LeetCode 55 — Jump Game
- LeetCode 45 — Jump Game II
- LeetCode 435 — Non-overlapping Intervals
- LeetCode 56 — Merge Intervals
- LeetCode 134 — Gas Station

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Dijkstra's shortest path algorithm is a greedy algorithm. Huffman coding (used in file compression like gzip) is a greedy algorithm. Operating system job scheduling (Shortest Job First) is greedy. Python's heapq-based priority queue powers greedy algorithms in network routing. |

## 3.15 Union Find (Disjoint Set Union)

### The Problem Situation

You need to efficiently group elements, check if two elements belong to the same group, or count the number of connected components. Union Find supports two operations — find (which group does this element belong to?) and union (merge two groups) — both in nearly O(1) with optimisations.

### Key Insight

| | |
|---|---|
| □ **Insight** | Every element starts as its own group. When you union two elements, you merge their groups by making one's root point to the other's root. When you find an element's |

| | |
|---|---|
| | group, you follow parent pointers to the root. Path compression flattens the tree during find. Union by rank keeps the tree shallow. |

## Analogy

| | |
|---|---|
| **Mental Model** | A network of islands. Each island starts disconnected. When you build a bridge between two islands, you merge their groups. Later, when asked 'are island A and island B connected?' you just check if they share the same group root — without tracing every bridge. |

## Basic Code Template

```
# Union Find — with path compression + union by rank
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank   = [0] * n
        self.count  = n   # number of components

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])  # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py: return False   # already connected
        # union by rank
        if self.rank[px] < self.rank[py]: px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        self.count -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 547 — Number of Provinces • LeetCode 684 — Redundant Connection • LeetCode 200 — Number of Islands |

## Example Walkthrough

```
# Example: Number of Provinces — n=3, connections=[[1,2],[1,3]]
# UF init: parent=[0,1,2], count=3
# union(0,1): find(0)=0, find(1)=1 → parent[1]=0, count=2
# union(0,2): find(0)=0, find(2)=2 → parent[2]=0, count=1
# Answer: count = 1 province

# Redundant Connection — find edge that creates a cycle
```

```
# Process edges one by one. If union() returns False,
# the two nodes are already connected → this edge is redundant.
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| find() with path compression | O(α(n)) ≈ O(1) | O(n) |
| union() with rank | O(α(n)) ≈ O(1) | |
| α(n) = inverse Ackermann — practically constant | | |

## Pattern Recognition Clues

- Keywords: connected components, union, same group, redundant edge, number of islands
- You are adding connections incrementally and querying connectivity
- Kruskal's MST algorithm uses Union Find to detect cycles
- Whenever you group elements dynamically — Union Find is the right tool

## Interview Traps

| | |
|---|---|
| □ **Trap** | Forgetting path compression — without it, find() degrades to O(n) |
| □ **Trap** | Using node values (1-indexed) but initialising parent with range(n) (0-indexed) — causes index error |
| □ **Trap** | Checking connected before union — union already returns False if already connected |

## Edge Cases

- Single node — it is its own component
- No edges — every node is its own component
- Self-loop edge — same node unioned with itself

## Important Practice Problems

- LeetCode 547 — Number of Provinces
- LeetCode 684 — Redundant Connection
- LeetCode 721 — Accounts Merge

- LeetCode 200 — Number of Islands (alternative approach)
- LeetCode 1319— Number of Operations to Make Network Connected

### Real World Python Usage

| | |
|---|---|
| **Real World** | Union Find powers Kruskal's MST algorithm used in network design. Social network friend grouping (finding all people in your social cluster) uses Union Find. Image segmentation algorithms use Union Find to group connected pixels. Hadoop's distributed data partitioning uses union-find-like grouping. |

## 3.16 Topological Sort

### The Problem Situation

You have a set of tasks with dependencies — task A must complete before task B. You need to find a valid ordering of all tasks, or detect if a circular dependency (cycle) makes ordering impossible. This is only defined for Directed Acyclic Graphs (DAGs).

### Key Insight

| | |
|---|---|
| **Insight** | A node with no incoming edges (in-degree = 0) has no prerequisites — it can be processed first. Process it, then remove its outgoing edges. Some other nodes will now have in-degree 0. Repeat. If all nodes are processed, a valid topological order exists. If some nodes remain, there is a cycle. |

### Analogy

| | |
|---|---|
| **Mental Model** | A university course schedule. You cannot take Algorithms before taking Data Structures. You cannot take Data Structures before Discrete Math. Topological sort gives you the order to take courses. If two courses are prerequisites of each other, no valid schedule exists — that is a cycle. |

### Basic Code Template

```
# Kahn's Algorithm — BFS-based topological sort
from collections import defaultdict, deque

def topo_sort(n, prerequisites):
    graph   = defaultdict(list)
    in_deg  = [0] * n
    for a, b in prerequisites:   # b must come before a
```

```
        graph[b].append(a)
        in_deg[a] += 1

    queue = deque([i for i in range(n) if in_deg[i] == 0])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph[node]:
            in_deg[nb] -= 1
            if in_deg[nb] == 0:
                queue.append(nb)

    return order if len(order) == n else []  # [] = cycle detected
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 207 — Course Schedule • LeetCode 210 — Course Schedule II • LeetCode 269 — Alien Dictionary |

## Alternative Template

```
# DFS-based topological sort
def topo_dfs(n, prerequisites):
    graph  = defaultdict(list)
    for a, b in prerequisites:
        graph[b].append(a)
    visited = [0] * n   # 0=unvisited, 1=visiting, 2=done
    order   = []
    def dfs(node):
        if visited[node] == 1: return False  # cycle!
        if visited[node] == 2: return True   # already processed
        visited[node] = 1
        for nb in graph[node]:
            if not dfs(nb): return False
        visited[node] = 2
        order.append(node)
        return True
    for i in range(n):
        if visited[i] == 0:
            if not dfs(i): return []
    return order[::-1]
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 210 — Course Schedule II • LeetCode 329 — Longest Increasing Path in Matrix |

## Example Walkthrough

```
# Example: 4 courses, prerequisites=[[1,0],[2,0],[3,1],[3,2]]
# Graph: 0→1, 0→2, 1→3, 2→3
# in_deg: [0,1,1,2]
```

```
# Queue start: [0] (only node with in_deg=0)
# Process 0: order=[0], reduce in_deg of 1,2 → in_deg=[0,0,0,2]
# Queue: [1,2]
# Process 1: order=[0,1], reduce in_deg of 3 → in_deg=[0,0,0,1]
# Process 2: order=[0,1,2], reduce in_deg of 3 → in_deg=[0,0,0,0]
# Queue: [3]
# Process 3: order=[0,1,2,3]
# len(order)==4==n → valid, return [0,1,2,3]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Kahn's BFS | O(V+E) | O(V+E) |
| DFS-based | O(V+E) | O(V+E) |

## Pattern Recognition Clues

- Keywords: prerequisites, dependencies, ordering, schedule, build order, compile order
- Any problem involving 'A must come before B' is topological sort
- Cycle detection in directed graphs — if topological sort fails, there is a cycle
- Alien dictionary — deduce character ordering using topological sort

## Interview Traps

| | |
|---|---|
| □ **Trap** | Building edges in the wrong direction — if [a,b] means 'b before a', then edge goes b→a |
| □ **Trap** | Returning a partial order when n != len(order) — check this condition to detect cycles |
| □ **Trap** | Using undirected graph — topological sort is only for directed graphs |

## Edge Cases

- No prerequisites — all nodes have in-degree 0, any order is valid
- Cycle in graph — impossible to complete, return empty
- Disconnected graph — must process all components

## Important Practice Problems

- LeetCode 207 — Course Schedule
- LeetCode 210 — Course Schedule II

- LeetCode 269 — Alien Dictionary
- LeetCode 329 — Longest Increasing Path in a Matrix
- LeetCode 310 — Minimum Height Trees

### Real World Python Usage

| | |
|---|---|
| **Real World** | Build systems (Make, Bazel, CMake) use topological sort to determine compilation order. Python's pip resolves package dependencies using topological sort. CI/CD pipeline stages use DAG scheduling with topological sort. Apache Airflow (workflow orchestration used in ML pipelines) executes tasks in topologically sorted order. |

## 3.17 Dijkstra's Algorithm

### The Problem Situation

You need the shortest path from a source node to all other nodes in a weighted graph where all edge weights are non-negative. BFS gives shortest path in unweighted graphs. Dijkstra extends BFS with a priority queue to handle edge weights.

### Key Insight

| | |
|---|---|
| **Insight** | Always expand the unvisited node with the smallest known distance first. When you reach a neighbour through this node, check if the new path is shorter than its currently known distance. If yes, update it. A min-heap efficiently finds the next closest node in O(log V). |

### Analogy

| | |
|---|---|
| **Mental Model** | GPS navigation finding the fastest route. You start at your location. The navigation system always explores the closest unvisited road junction next. When it finds a shorter path to a junction via a different route, it updates the best known time. It never revisits a junction once finalised. |

### Basic Code Template

```
# Dijkstra with heapq — O((V+E) log V)
import heapq
from collections import defaultdict

def dijkstra(graph, start, n):
    dist = [float('inf')] * n
```

```
    dist[start] = 0
    heap = [(0, start)]   # (distance, node)
    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]: continue  # outdated entry
        for neighbour, weight in graph[node]:
            new_dist = dist[node] + weight
            if new_dist < dist[neighbour]:
                dist[neighbour] = new dist
                heapq.heappush(heap, (new_dist, neighbour))
    return dist

# Build weighted graph
graph = defaultdict(list)
edges = [(0,1,4),(0,2,1),(2,1,2),(1,3,1),(2,3,5)]
for u, v, w in edges:
    graph[u].append((v, w))
    graph[v].append((u, w))  # undirected

print(dijkstra(graph, 0, 4))  # [0,3,1,4]
```

| | |
|---|---|
| **✓This Template Solves** | LeetCode 743 — Network Delay Time • LeetCode 1631 — Path with Minimum Effort • LeetCode 778 — Swim in Rising Water |

## Example Walkthrough

```
# Example: graph=[[0,1,4],[0,2,1],[2,1,2],[1,3,1]]
# dist=[0,inf,inf,inf], heap=[(0,0)]
# Pop (0,0): neighbours (1,4),(2,1)
#   dist[1]=4 → heap=[(1,2),(4,1)]
#   dist[2]=1 → heap=[(1,2),(4,1)]
# Pop (1,2): neighbours (0,1),(1,2),(3,5)
#   0: 1+1=2 > dist[0]=0, skip
#   1: 1+2=3 < dist[1]=4 → dist[1]=3, push (3,1)
#   3: 1+5=6, dist[3]=6, push (6,3)
# Pop (3,1): neighbours (0,4),(2,2),(3,1)
#   dist[3]=3+1=4 → push (4,3)
# Final dist=[0,3,1,4]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Time | O((V+E) log V) | O(V+E) |
| With Fibonacci heap | O(E + V log V) | |

## Pattern Recognition Clues

- Keywords: shortest path, weighted graph, minimum cost, network delay, cheapest flight
- Edge weights are non-negative — must use Dijkstra, not BFS
- If edges have negative weights — use Bellman-Ford instead
- Finding path in a grid where movement cost varies — Dijkstra on grid

### Interview Traps

| | |
|---|---|
| □ **Trap** | Not skipping stale heap entries — if d > dist[node]: continue is essential |
| □ **Trap** | Using Dijkstra with negative weights — gives wrong answer. Use Bellman-Ford instead. |
| □ **Trap** | Forgetting to initialise dist = [float('inf')] * n before setting dist[start] = 0 |

### Edge Cases

- Disconnected graph — some nodes have dist = infinity at the end
- Single node — dist = [0]
- Target not reachable — return infinity or -1

### Important Practice Problems

- LeetCode 743 — Network Delay Time
- LeetCode 1631 — Path With Minimum Effort
- LeetCode 787 — Cheapest Flights Within K Stops
- LeetCode 778 — Swim in Rising Water
- LeetCode 1514 — Path with Maximum Probability

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Google Maps and Apple Maps use variants of Dijkstra for route planning. Internet routing protocols (OSPF) use Dijkstra to find shortest network paths. Game AI pathfinding uses A* (an informed version of Dijkstra). Python's networkx library provides dijkstra_path() built on this exact algorithm. |

## 3.18 Interval Problems

## The Problem Situation

You are given a list of intervals [start, end] and need to merge overlapping intervals, count minimum resources needed, insert a new interval, or find the maximum overlap. Sorting intervals by start time almost always unlocks the greedy or linear solution.

## Key Insight

| | |
|---|---|
| **Insight** | Sort by start time. Two intervals [a,b] and [c,d] overlap if c <= b (the second starts before the first ends). When they overlap, merge by extending the end: max(b, d). For minimum resources (meeting rooms), use a min-heap tracking end times of active intervals. |

## Analogy

| | |
|---|---|
| **Mental Model** | A calendar with bookings. Sort appointments by start time. If the next appointment starts before the current one ends, they conflict (overlap). To merge overlapping bookings, extend the current booking to the later end time. |

## Basic Code Template

```
# Merge intervals
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:              # overlap
            merged[-1][1] = max(merged[-1][1], end)
        else:                                   # no overlap
            merged.append([start, end])
    return merged
```

| | |
|---|---|
| **This Template Solves** | LeetCode 56 — Merge Intervals • LeetCode 57 — Insert Interval |

## Alternative Template

```
# Minimum meeting rooms (minimum resources for overlapping intervals)
import heapq
def min_meeting_rooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []   # min-heap of end times
    for start, end in intervals:
        if heap and heap[0] <= start:
            heapq.heapreplace(heap, end)  # reuse room
        else:
            heapq.heappush(heap, end)     # new room needed
```

```
    return len(heap)

# Insert interval into sorted non-overlapping list
def insert(intervals, new_interval):
    result, i, n = [], 0, len(intervals)
    while i < n and intervals[i][1] < new_interval[0]:
        result.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= new_interval[1]:
        new_interval[0] = min(new_interval[0], intervals[i][0])
        new_interval[1] = max(new_interval[1], intervals[i][1])
        i += 1
    result.append(new_interval)
    return result + intervals[i:]
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 253 — Meeting Rooms II • LeetCode 57 — Insert Interval • LeetCode 435 — Non-overlapping Intervals |

## Example Walkthrough

```
# Example: merge [[1,3],[2,6],[8,10],[15,18]]
# Sort: already sorted by start
# merged=[[1,3]]
# [2,6]: 2<=3 → overlap, extend: merged=[[1,6]]
# [8,10]: 8>6 → no overlap, append: merged=[[1,6],[8,10]]
# [15,18]: 15>10 → append: merged=[[1,6],[8,10],[15,18]]
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| Merge intervals | O(n log n) | O(n) |
| Meeting rooms | O(n log n) | O(n) |
| Insert interval | O(n) | O(n) |

## Pattern Recognition Clues

- Keywords: intervals, meetings, overlap, merge, schedule, rooms needed
- Sorting by start time is almost always the first step in interval problems
- Minimum rooms needed = maximum number of overlapping intervals at any point
- Non-overlapping count = total - minimum removals (greedy by end time)

## Interview Traps

| | |
|---|---|
| □ **Trap** | Merging condition: overlap is start <= prev_end, NOT start < prev_end (touching intervals overlap) |
| □ **Trap** | Forgetting to sort — interval problems almost never work without sorting first |
| □ **Trap** | Updating end with max but not handling the case where new interval is completely inside old |

### Edge Cases

- Single interval — return as is
- No overlapping intervals — return original list
- New interval inserted at the beginning or end
- New interval completely inside an existing interval

### Important Practice Problems

- LeetCode 56 — Merge Intervals
- LeetCode 57 — Insert Interval
- LeetCode 253 — Meeting Rooms II
- LeetCode 435 — Non-overlapping Intervals
- LeetCode 452 — Minimum Number of Arrows to Burst Balloons

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Calendar systems (Google Calendar) detect and merge overlapping bookings using interval merging. Database query optimisers merge overlapping index scans. Network bandwidth allocation uses interval scheduling. Python's datetime interval arithmetic uses overlap detection logic. |

## 3.19 Matrix Traversal

---

### The Problem Situation

You are given a 2D grid and need to explore connected regions, find shortest paths, count islands, or traverse in a specific pattern (spiral, diagonal). Matrix problems are essentially graph problems where each cell is a node and adjacent cells are edges.

### Key Insight

| | |
|---|---|
| □ **Insight** | Treat each cell as a node. Movement in 4 directions (up/down/left/right) creates edges. Use DFS for connected region exploration (flood fill, island counting). Use BFS for shortest path in the grid. Always check bounds before visiting a cell. |

## Analogy

| | |
|---|---|
| □ **Mental Model** | A city grid of streets. Each intersection is a cell. You can travel North, South, East, or West from any intersection. Blocked cells (walls) are not traversable. Finding the shortest route from one corner to another is BFS on the grid. |

## Basic Code Template

```
# 4-directional DFS — flood fill / island counting
def num_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    def dfs(r, c):
        if r<0 or r>=rows or c<0 or c>=cols or grid[r][c]!='1':
            return
        grid[r][c] = '0'   # mark visited (in-place)
        dfs(r+1,c); dfs(r-1,c); dfs(r,c+1); dfs(r,c-1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                dfs(r, c)
                count += 1
    return count
```

| | |
|---|---|
| ✅**This Template Solves** | LeetCode 200 — Number of Islands • LeetCode 695 — Max Area of Island • LeetCode 733 — Flood Fill |

## Alternative Template

```
# BFS shortest path on grid
from collections import deque
def shortest_path(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    dirs    = [(0,1),(0,-1),(1,0),(-1,0)]
    visited = set([start])
    queue   = deque([(start[0], start[1], 0)])
    while queue:
        r, c, dist = queue.popleft()
        if (r,c) == end: return dist
        for dr, dc in dirs:
            nr, nc = r+dr, c+dc
            if (0<=nr<rows and 0<=nc<cols and
                grid[nr][nc]!=1 and (nr,nc) not in visited):
```

```
                visited.add((nr,nc))
                queue.append((nr, nc, dist+1))
    return -1

# Spiral order traversal
def spiral_order(matrix):
    result = []
    while matrix:
        result += matrix.pop(0)             # top row
        if matrix and matrix[0]:            # right column
            for row in matrix: result.append(row.pop())
        if matrix: result += matrix.pop()[::-1]  # bottom row
        if matrix and matrix[0]:            # left column
            for row in matrix[::-1]: result.append(row.pop(0))
    return result
```

| | |
|---|---|
| **✅This Template Solves** | LeetCode 994 — Rotting Oranges • LeetCode 1091 — Shortest Path in Binary Matrix • LeetCode 54 — Spiral Matrix |

## Example Walkthrough

```
# Always define directions array — cleaner than 4 if-statements
dirs = [(0,1),(0,-1),(1,0),(-1,0)]   # right, left, down, up
# For 8-directional (including diagonals):
dirs = [(dr,dc) for dr in [-1,0,1] for dc in [-1,0,1] if (dr,dc)!=(0,0)]

# Bounds check pattern
for dr, dc in dirs:
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols:
        # safe to access grid[nr][nc]
        pass
```

## Time & Space Complexity

| Case | Time | Space |
|---|---|---|
| DFS on grid | O(rows × cols) | O(rows × cols) |
| BFS shortest path | O(rows × cols) | O(rows × cols) |
| Spiral traversal | O(rows × cols) | O(1) |

## Pattern Recognition Clues

- Keywords: grid, matrix, island, region, connected, shortest path in grid, flood fill
- Count connected regions — DFS/BFS from each unvisited cell
- Shortest path in grid — always BFS (not DFS)

- Multi-source spread (rotting oranges) — multi-source BFS

### Interview Traps

| | |
|---|---|
| □ **Trap** | Forgetting bounds check — always check 0<=nr<rows and 0<=nc<cols before accessing grid[nr][nc] |
| □ **Trap** | Modifying the input grid to mark visited — may be forbidden; use a separate visited set instead |
| □ **Trap** | Using DFS for shortest path in grid — DFS does not guarantee shortest. Use BFS. |

### Edge Cases

- Empty grid
- Grid with a single cell
- No path exists between start and end
- Grid boundary cells — must not go out of bounds

### Important Practice Problems

- LeetCode 200 — Number of Islands
- LeetCode 994 — Rotting Oranges
- LeetCode 1091 — Shortest Path in Binary Matrix
- LeetCode 54 — Spiral Matrix
- LeetCode 695 — Max Area of Island

### Real World Python Usage

| | |
|---|---|
| □ **Real World** | Game development uses grid BFS for pathfinding (A* is an optimised BFS). Robotics uses grid DFS/BFS for maze-solving and navigation. Image processing uses flood fill (grid DFS) for region detection. Geographic information systems (GIS) use grid traversal for terrain analysis. Python's scipy.ndimage uses grid-connected-component algorithms. |

## Pattern Comparison — Section 3B

| Pattern | Best For | Core Idea | Key Tool |
|---|---|---|---|
| Backtracking | All combos/subsets/perms | Choose→explore→unchoose | Recursion + list |
| Divide & Conquer | Sorting, tree problems | Split, solve, merge | Recursion + @lru_cache |
| DP (Top-Down) | Overlapping subproblems | Memoize recursive calls | @lru_cache |
| DP (Bottom-Up) | Optimal substructure | Fill table from base case up | Array / 2D array |
| Greedy | Interval, scheduling, jumps | Locally optimal = globally optimal | Sort + one pass |
| Union Find | Connected components, groups | find() + union() in near O(1) | parent[] + rank[] |
| Topological Sort | Dependency ordering | Process 0 in-degree nodes first | deque + in_deg[] |
| Dijkstra | Shortest path (weighted) | Min-heap → expand closest first | heapq |
| Intervals | Overlap, merge, scheduling | Sort by start, merge or count | Sort + heap |
| Matrix Traversal | Grid regions, shortest grid path | DFS flood fill / BFS shortest | visited set + dirs[] |

# Master Pattern Recognition Guide

When you read a problem, scan for these keywords and map them to the correct pattern instantly.

| If you see this… | Think this pattern |
|---|---|
| sorted array + pair/triplet | Two Pointers |
| subarray / substring / contiguous | Sliding Window or Prefix Sum |
| sum equals k / range sum | Prefix Sum + Hash Map |
| frequency / count / duplicate / anagram | Hash Map Counting |
| sorted + find target / boundary | Binary Search |
| next greater / smaller / histogram | Monotonic Stack |
| sliding window max/min | Monotonic Queue |
| linked list cycle / middle | Fast and Slow Pointers |

| | |
|---|---|
| all paths / explore / subtree | DFS |
| shortest path (unweighted) / level order | BFS |
| all combinations / subsets / permutations | Backtracking |
| minimum/maximum / count ways / optimal | Dynamic Programming |
| interval / merge / schedule / rooms | Greedy + Intervals |
| connected components / union / group | Union Find |
| prerequisites / dependencies / ordering | Topological Sort |
| shortest path (weighted graph) | Dijkstra |
| grid / island / region / matrix | Matrix DFS or BFS |

---

**End of Section 3B.** Continue with Section 4 — Pattern Recognition System.

# Python & Algorithms for Coding Interviews

Section 4 — Pattern Recognition System

Reading a problem and immediately knowing which pattern to apply is the single most valuable skill in coding interviews. This section gives you a systematic, repeatable process to do exactly that — every time.

## 4.1 Problem → Pattern Keyword Map

These are the exact words and phrases that appear in LeetCode problems. When you spot them, your brain should immediately fire the associated pattern.

### Array and String Keywords

| Keyword / Phrase in Problem | → Pattern to Use |
|---|---|
| sorted array, find pair/triplet, sum to target | Two Pointers |
| subarray, substring, contiguous, window of size k | Sliding Window |
| longest/shortest subarray satisfying condition | Sliding Window (variable) |
| range sum, sum of subarray, running total | Prefix Sum |
| subarray sum equals k, count subarrays with sum | Prefix Sum + Hash Map |
| frequency, count, anagram, duplicate, most common | Hash Map / Counter |
| two sum, find complement, find pair | Hash Map |
| sorted array, find target, search | Binary Search |
| minimum/maximum satisfying a condition (answer range) | Binary Search on Answer Space |
| next greater, next smaller, previous larger | Monotonic Stack |

| | |
|---|---|
| largest rectangle, histogram, trapping water | Monotonic Stack |
| sliding window max/min, maximum in every window | Monotonic Queue (Deque) |

## Linked List Keywords

| Keyword / Phrase in Problem | → Pattern to Use |
|---|---|
| cycle, loop, circular linked list | Fast and Slow Pointers |
| middle of linked list | Fast and Slow Pointers |
| palindrome linked list | Fast and Slow Pointers + Reverse |
| reverse a linked list | Iterative with prev/curr/next |
| merge two sorted lists | Two Pointers on linked lists |
| remove Nth node from end | Two Pointers (gap of N) |
| LRU Cache | Doubly Linked List + Hash Map |

## Tree and Graph Keywords

| Keyword / Phrase in Problem | → Pattern to Use |
|---|---|
| all paths, explore, subtree, depth, height | DFS |
| lowest common ancestor, path between nodes | DFS |
| level order, shortest path (unweighted), nearest | BFS |
| connected components, number of islands, regions | DFS or Union Find |
| shortest path, minimum cost (weighted graph) | Dijkstra |
| prerequisites, dependencies, build order, schedule | Topological Sort |
| cycle in directed graph | Topological Sort or DFS |
| union, same group, connected, redundant edge | Union Find |
| autocomplete, prefix search, word search grid | Trie |

## Optimisation and Enumeration Keywords

| Keyword / Phrase in Problem | → Pattern to Use |
|---|---|
| minimum/maximum value, optimal, count ways | Dynamic Programming |

| | |
|---|---|
| can we achieve X, is it possible | DP or Greedy |
| overlapping subproblems (same inputs computed twice) | DP with Memoisation |
| all combinations, all subsets, generate all | Backtracking |
| all permutations, all arrangements | Backtracking |
| N-Queens, Sudoku, word search on grid | Backtracking |
| intervals, meetings, overlap, merge, schedule | Greedy + Sort by time |
| minimum rooms, maximum non-overlapping | Greedy + Heap |
| jump game, minimum steps, greedy choice | Greedy |

## 4.2 Input Structure → Pattern Map

| Input Type | First Thought | Common Patterns |
|---|---|---|
| 1D sorted array | Binary search? Two pointers? | Binary Search, Two Pointers, Sliding Window |
| 1D unsorted array | Sort first? Hash map? DP? | Hash Map, Prefix Sum, Sliding Window, DP |
| String | Character frequency? Substring? | Sliding Window, Hash Map, Two Pointers, Trie |
| 2D matrix / grid | Regions? Shortest path? | DFS, BFS, Union Find, DP on grid |
| Linked list | Cycle? Middle? Reverse? | Fast/Slow Pointers, Two Pointers |
| Binary tree | Subtree result? Level-by-level? | DFS (recursion), BFS (level order) |
| Graph (edge list) | Shortest path? Components? | BFS, DFS, Dijkstra, Union Find, Topo Sort |
| Intervals [s,e] | Sort by start. Overlap? | Greedy, Sort, Heap |
| Numbers (count ways) | Overlapping subproblems? | DP (top-down or bottom-up) |
| Strings (generate) | All combinations/permutations? | Backtracking |

## 4.3 Constraint-Based Pattern Detection

The size of n reveals what time complexity is required — and that directly tells you which patterns are viable.

| Input Size (n) | Maximum Viable Complexity | Patterns That Fit |
|---|---|---|
| n ≤ 10 | O(n!) or O(2^n) | Brute force, Backtracking all permutations |
| n ≤ 20 | O(2^n) | Backtracking, Bitmask DP |
| n ≤ 100 | $O(n^3)$ | 3-nested loop DP, Floyd-Warshall |
| n ≤ 1,000 | $O(n^2)$ | Nested loops, $O(n^2)$ DP |
| n ≤ 10,000 | $O(n^2)$ borderline | Careful $O(n^2)$ — prefer O(n log n) |
| n ≤ 100,000 | O(n log n) | Sorting, Binary Search, Heap, Divide & Conquer |
| n ≤ 1,000,000 | O(n) | Two Pointers, Sliding Window, Hash Map, Prefix Sum |
| n > 10,000,000 | O(n) or O(log n) | Binary Search, Counting Sort, one-pass algorithms |

| | |
|---|---|
| **Key Rule** | Look at constraints first. If n ≤ 100,000 you need O(n log n) or better. If n ≤ 1,000,000 you need O(n). This immediately rules out $O(n^2)$ DP or O(2^n) backtracking. |

## 4.4 Pattern Decision Flowchart

Work through these questions in order when you read a new problem. Stop at the first question that gives you a clear answer.

**Step 1:** Is the input a linked list?
→ **Fast/Slow Pointers (cycle/middle) OR Two Pointers (merge/remove)**

**Step 2:** Is it a tree or binary tree?
→ **DFS for subtree computation OR BFS for level-order / shortest path**

**Step 3:** Is it a graph (edges, connections)?
→ **BFS (shortest unweighted) / Dijkstra (weighted) / Topo Sort (dependencies) / Union Find (groups)**

**Step 4:** Is it a 2D grid / matrix?
→ **DFS for regions/islands OR BFS for shortest path on grid**

**Step 5:** Is the input sorted (or can be sorted)?
→ **Two Pointers OR Binary Search OR Greedy (intervals)**

**Step 6:** Does it involve contiguous elements?

→ **Sliding Window OR Prefix Sum**

**Step 7:** Does it need next greater/smaller?
→ **Monotonic Stack**

**Step 8:** Does it count frequencies or need O(1) lookup?
→ **Hash Map / Counter / Set**

**Step 9:** Does it ask for ALL combinations/subsets/perms?
→ **Backtracking**

**Step 10:** Does it ask for MINIMUM / MAXIMUM / COUNT WAYS?
→ **Dynamic Programming**

**Step 11:** Is there a greedy choice property?
→ **Greedy**

**Step 12:** Does it group elements or detect components?
→ **Union Find**

**Step 13:** Does it involve prefix matching or word search?
→ **Trie**

| | |
|---|---|
| **Pro Tip** | Most interview problems are answered by steps 1–6. The deeper steps handle more specialised problems. |

## 4.5 Pre-Coding Mental Checklist

Run through this checklist before writing a single line of code.

### Step 1 — Understand the Problem (2 min)

- Read the problem twice. Do not skim.
- Identify: What is the INPUT? What is the OUTPUT?
- What are the CONSTRAINTS? (size of n, value ranges, sorted or not)
- What should I return if the input is empty or has one element?
- Can elements be negative? Can there be duplicates? Is the array sorted?

### Step 2 — Work Through Examples (2 min)

- Trace through the provided examples manually.
- Create your own small example (3–5 elements) and solve it by hand.
- Think about edge cases: empty, single element, all same, all negative.
- Ask yourself: Can I see a pattern in how I solved the example by hand?

### Step 3 — Identify the Pattern (1 min)

- What is the input structure? (array, string, tree, graph, grid, linked list)
- What does the problem ask? (min/max, count, find all, shortest, yes/no)
- What are the constraints? (n size → required complexity)
- Scan for keywords → match to pattern using Sections 4.1 and 4.2
- No keyword match? Start brute force, then optimise.

### Step 4 — Plan Before Coding (1 min)

- State the approach: 'I will use sliding window because...'
- Identify the state: What variables do I need to track?
- Identify the transition: How does the state change at each step?
- Estimate complexity: Time and Space — does it fit the constraints?
- Only start coding once you have a clear plan.

### Step 5 — Code and Test (10–15 min)

- Write clean code with simple variable names (left, right, slow, fast, seen).
- Handle the base case / edge case first.
- Test with your small example by tracing through the code manually.
- Check for off-by-one errors (< vs <=, i vs i+1).
- Check for empty input, single element, and edge cases from Step 2.

## 4.6 Brute Force → Optimise Framework

When you cannot immediately identify the pattern, start with brute force. Then apply these strategies one by one.

| Brute Force Issue | Optimisation Strategy | Result |
|---|---|---|
| Repeated scanning of array for lookup | Replace list scan with Hash Map | O(n) → O(1) per lookup |
| Recomputing sum of every subarray | Use Prefix Sum array | $O(n^2) \rightarrow O(n)$ |

| | | |
|---|---|---|
| Checking all pairs in sorted array | Use Two Pointers | $O(n^2) \rightarrow O(n)$ |
| Checking all subarrays of size k | Use Sliding Window | $O(nk) \rightarrow O(n)$ |
| Linear scan for sorted data | Use Binary Search | $O(n) \rightarrow O(\log n)$ |
| Recursive tree with repeated calls | Add Memoisation (@lru_cache) | $O(2^n) \rightarrow O(n)$ |
| Checking all subsets | Use DP bottom-up table | $O(2^n) \rightarrow O(n \cdot target)$ |
| Finding max in every window | Use Monotonic Deque | $O(nk) \rightarrow O(n)$ |
| Union/find on naive structures | Union Find with path compression | $O(n) \rightarrow O(\alpha(n)) \approx O(1)$ |

---

**End of Section 4.** Continue with Section 5 — Universal Code Templates.

# Python & Algorithms for Coding Interviews

Section 5 — Universal Code Templates

Every template here is short enough to memorise. Each one is a battle-tested starting point — adapt variable names and logic to the specific problem, but the skeleton stays the same.

## 5.1 DFS — Recursive (Tree & Graph)

```
# Tree DFS
def dfs_tree(root):
    if not root:
        return 0   # adjust return type per problem
    left  = dfs_tree(root.left)
    right = dfs_tree(root.right)
    return 1 + max(left, right)   # example: tree height

# Graph DFS
def dfs_graph(graph, node, visited):
    visited.add(node)
    for nb in graph[node]:
        if nb not in visited:
            dfs_graph(graph, nb, visited)

# Outer loop for disconnected graphs
visited = set()
for node in range(n):
    if node not in visited:
        dfs_graph(graph, node, visited)
```

| | |
|---|---|
| ✓ **Solves** | LeetCode 104 Max Depth • 200 Number of Islands • 543 Diameter • 133 Clone Graph |

## 5.2 DFS — Iterative (Explicit Stack)

```
def dfs_iterative(graph, start):
    visited = set()
    stack   = [start]
    while stack:
```

```
        node = stack.pop()
        if node in visited: continue
        visited.add(node)
        for nb in graph[node]:
            if nb not in visited:
                stack.append(nb)

# Use when recursion depth might exceed Python's 1000 frame limit
```

| ✓Solves | LeetCode 200 Number of Islands • 695 Max Area • Any deep graph problem |
|---|---|

## 5.3 BFS — Standard & Multi-Source

```
from collections import deque

# Standard BFS — shortest path
def bfs(graph, start):
    visited = set([start])
    queue   = deque([(start, 0)])   # (node, distance)
    while queue:
        node, dist = queue.popleft()
        for nb in graph[node]:
            if nb not in visited:
                visited.add(nb)
                queue.append((nb, dist + 1))

# Level-order tree BFS
def level_order(root):
    if not root: return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):   # one level at a time
            node = queue.popleft()
            level.append(node.val)
            if node.left:  queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result

# Multi-source BFS (grid) — enqueue ALL sources at start
def multi_bfs(grid):
    rows, cols = len(grid), len(grid[0])
    queue, visited = deque(), set()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:   # source condition
                queue.append((r,c,0)); visited.add((r,c))
    dirs = [(0,1),(0,-1),(1,0),(-1,0)]
    while queue:
        r,c,dist = queue.popleft()
        for dr,dc in dirs:
            nr,nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and (nr,nc) not in visited:
```

```
                visited.add((nr,nc)); queue.append((nr,nc,dist+1))
```

| | |
|---|---|
| ✓Solves | LeetCode 102 Level Order • 127 Word Ladder • 994 Rotting Oranges • 542 01 Matrix |

## 5.4 Binary Search — 3 Variants

```
# Variant 1: Find exact target
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:              # note: <=
        mid = left + (right - left) // 2
        if   nums[mid] == target: return mid
        elif nums[mid] <  target: left  = mid + 1
        else:                     right = mid - 1
    return -1

# Variant 2: Left boundary (first index where nums[i] >= target)
def left_bound(nums, target):
    left, right = 0, len(nums)        # note: right = len
    while left < right:               # note: <
        mid = (left + right) // 2
        if nums[mid] < target: left  = mid + 1
        else:                  right = mid
    return left

# Variant 3: Binary search on answer space
def search_on_answer(nums, k):
    left, right = min(nums), sum(nums)  # define answer range
    while left < right:
        mid = (left + right) // 2
        if feasible(nums, k, mid): right = mid
        else:                      left  = mid + 1
    return left
```

| | |
|---|---|
| ✓Solves | LeetCode 704 • 278 First Bad Version • 875 Koko Eating Bananas • 410 Split Array Largest Sum |

## 5.5 Backtracking — Subsets / Combos / Permutations

```
# Subsets
def subsets(nums):
    result = []
    def bt(start, path):
        result.append(list(path))    # record at every node
        for i in range(start, len(nums)):
            path.append(nums[i]); bt(i+1, path); path.pop()
    bt(0, []); return result
```

```
# Combinations (choose k from n)
def combine(n, k):
    result = []
    def bt(start, path):
        if len(path)==k: result.append(list(path)); return
        for i in range(start, n+1):
            path.append(i); bt(i+1, path); path.pop()
    bt(1, []); return result

# Permutations
def permutations(nums):
    result = []
    def bt(path, remaining):
        if not remaining: result.append(list(path)); return
        for i in range(len(remaining)):
            path.append(remaining[i])
            bt(path, remaining[:i]+remaining[i+1:])
            path.pop()
    bt([], nums); return result

# Skip duplicates (sort first + skip same siblings)
nums.sort()
if i > start and nums[i] == nums[i-1]: continue
```

| | |
|---|---|
| ✓Solves | LeetCode 78 Subsets • 77 Combinations • 46 Permutations • 90 Subsets II • 39 Combination Sum |

## 5.6 DP — Top-Down Memoisation

```
from functools import lru_cache

# 1D DP
@lru_cache(maxsize=None)
def dp_1d(i):
    if i <= 0: return BASE_CASE
    return f(dp_1d(i-1), dp_1d(i-2))   # adjust recurrence

# 2D DP
@lru_cache(maxsize=None)
def dp_2d(i, j):
    if i==0 or j==0: return BASE_CASE
    if s1[i-1]==s2[j-1]: return dp_2d(i-1, j-1) + 1
    return max(dp_2d(i-1, j), dp_2d(i, j-1))

# Note: list inputs must be converted to tuple for hashability
@lru_cache(maxsize=None)
def dp_coins(amount, coins):  # coins must be tuple
    if amount==0: return 0
    if amount <0: return float('inf')
    return 1 + min(dp_coins(amount-c, coins) for c in coins)

dp_2d.cache_clear()   # reset between test cases if needed
```

| | |
|---|---|
| ✓Solves | LeetCode 70 Climbing Stairs • 322 Coin Change • 1143 LCS • 139 Word Break |

## 5.7 DP — Bottom-Up Tabulation

```
# 1D DP
def dp_1d_tab(n):
    dp    = [0] * (n + 1)
    dp[0] = BASE_0
    dp[1] = BASE_1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]   # adjust recurrence
    return dp[n]

# 2D DP (LCS / Edit Distance)
def dp_2d_tab(s1, s2):
    m, n = len(s1), len(s2)
    dp   = [[0]*(n+1) for _ in range(m+1)]
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1]==s2[j-1]: dp[i][j] = dp[i-1][j-1] + 1
            else:                dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

# Space-optimised rolling array
def dp_rolling(s1, s2):
    n  = len(s2)
    dp = [0]*(n+1)
    for ch in s1:
        prev = 0
        for j in range(1, n+1):
            temp  = dp[j]
            dp[j] = prev+1 if ch==s2[j-1] else max(dp[j], dp[j-1])
            prev  = temp
    return dp[n]
```

| | |
|---|---|
| ✓Solves | LeetCode 322 Coin Change • 1143 LCS • 300 LIS • 72 Edit Distance • 416 Partition Equal Subset Sum |

## 5.8 Union Find (DSU)

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank   = [0] * n
        self.count  = n

    def find(self, x):                        # path compression
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
```

```
    def union(self, x, y):                    # union by rank
        px, py = self.find(x), self.find(y)
        if px == py: return False
        if self.rank[px] < self.rank[py]: px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]: self.rank[px] += 1
        self.count -= 1
        return True

    def connected(self, x, y):
        return self.find(x) == self.find(y)

uf = UnionFind(n)
for u, v in edges: uf.union(u, v)
print(uf.count)   # number of connected components
```

| ✓Solves | LeetCode 547 Number of Provinces • 684 Redundant Connection • 721 Accounts Merge |
|---|---|

## 5.9 Trie (Prefix Tree)

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end   = False

class Trie:
    def __init__(self): self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
        node.is_end = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children: return False
            node = node.children[ch]
        return node.is_end

    def starts_with(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children: return False
            node = node.children[ch]
        return True
```

| | |
|---|---|
| ✓Solves | LeetCode 208 Implement Trie • 212 Word Search II • 648 Replace Words • 677 Map Sum Pairs |

## 5.10 Graph Build + Traversal

```
from collections import defaultdict

# Unweighted undirected
graph = defaultdict(list)
for u, v in edges:
    graph[u].append(v)
    graph[v].append(u)   # omit for directed

# Weighted
for u, v, w in weighted_edges:
    graph[u].append((v, w))
    graph[v].append((u, w))

# Count connected components
visited, count = set(), 0
def dfs(node):
    visited.add(node)
    for nb in graph[node]:
        if nb not in visited: dfs(nb)
for i in range(n):
    if i not in visited:
        dfs(i); count += 1
```

| | |
|---|---|
| ✓Solves | LeetCode 200 Number of Islands • 547 Number of Provinces • 323 Number of Connected Components |

## 5.11 Heap — Min / Max / Priority Queue

```
import heapq

heap = []
heapq.heappush(heap, val)          # O(log n)
smallest = heapq.heappop(heap)     # O(log n)
peek     = heap[0]                 # O(1)
heapq.heapify(list_of_nums)        # O(n)

# Max-heap — negate values
heapq.heappush(max_heap, -val)
largest = -heapq.heappop(max_heap)

# Priority queue with metadata
heapq.heappush(heap, (priority, data))
priority, data = heapq.heappop(heap)
```

```
# Top K in a stream — maintain size-k min-heap
for num in stream:
    heapq.heappush(heap, num)
    if len(heap) > k: heapq.heappop(heap)
# heap now contains the k largest
```

| | |
|---|---|
| ✓Solves | LeetCode 215 Kth Largest • 347 Top K Frequent • 295 Find Median from Stream • 743 Network Delay |

## 5.12 Sliding Window — Fixed & Variable

```
# Fixed window
def fixed window(nums, k):
    window_sum = sum(nums[:k])
    result     = [window_sum]
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i-k]
        result.append(window_sum)
    return result

# Variable window — longest satisfying condition
def variable_window(s):
    left, best, state = 0, 0, {}
    for right, ch in enumerate(s):
        state[ch] = state.get(ch, 0) + 1   # expand
        while invalid(state):               # shrink
            state[s[left]] -= 1
            if state[s[left]] == 0: del state[s[left]]
            left += 1
        best = max(best, right - left + 1)  # record
    return best
```

| | |
|---|---|
| ✓Solves | LeetCode 3 Longest Substring • 76 Minimum Window Substring • 567 Permutation in String |

## 5.13 Two Pointers

```
# Opposite ends — sorted array
def two_ptr_opposite(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        total = nums[left] + nums[right]
        if   total == target: return [left, right]
        elif total <  target: left  += 1
        else:                 right -= 1
    return []

# Same direction — partition / remove
def two_ptr_same(nums):
```

```
    slow = 0
    for fast in range(len(nums)):
        if condition(nums[fast]):
            nums[slow] = nums[fast]
            slow += 1
    return slow
```

| | |
|---|---|
| ✓Solves | LeetCode 167 Two Sum II • 15 3Sum • 11 Container With Water • 26 Remove Duplicates |

## 5.14 Monotonic Stack

```
# Next Greater Element — decreasing stack
def next_greater(nums):
    result = [-1] * len(nums)
    stack  = []
    for i, num in enumerate(nums):
        while stack and nums[stack[-1]] < num:
            result[stack.pop()] = num
        stack.append(i)
    return result

# Largest Rectangle in Histogram — increasing stack
def largest_rect(heights):
    stack, max_area = [], 0
    heights = heights + [0]   # sentinel to flush stack
    for i, h in enumerate(heights):
        start = i
        while stack and heights[stack[-1]] > h:
            idx      = stack.pop()
            width    = i - (stack[-1] if stack else -1) - 1
            max_area = max(max_area, heights[idx]*width)
            start    = idx
        stack.append(start)
    return max_area
```

| | |
|---|---|
| ✓Solves | LeetCode 739 Daily Temperatures • 496 Next Greater • 84 Largest Rectangle • 42 Trapping Rain Water |

## 5.15 Topological Sort — Kahn's BFS

```
from collections import defaultdict, deque

def topo_sort(n, prerequisites):
    graph  = defaultdict(list)
    in_deg = [0] * n
    for a, b in prerequisites:   # b must come before a
        graph[b].append(a)
        in_deg[a] += 1
```

```
    queue = deque(i for i in range(n) if in deg[i] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nb in graph[node]:
            in_deg[nb] -= 1
            if in_deg[nb] == 0: queue.append(nb)

    return order if len(order)==n else []  # [] = cycle detected
```

| ✓Solves | LeetCode 207 Course Schedule • 210 Course Schedule II • 269 Alien Dictionary |
|---|---|

## 5.16 Dijkstra's Shortest Path

```
import heapq
from collections import defaultdict

def dijkstra(n, edges, start):
    graph = defaultdict(list)
    for u, v, w in edges:
        graph[u].append((v, w))
        graph[v].append((u, w))   # undirected

    dist = [float('inf')] * n
    dist[start] = 0
    heap = [(0, start)]

    while heap:
        d, node = heapq.heappop(heap)
        if d > dist[node]: continue  # stale entry — skip
        for nb, w in graph[node]:
            nd = dist[node] + w
            if nd < dist[nb]:
                dist[nb] = nd
                heapq.heappush(heap, (nd, nb))

    return dist
```

| ✓Solves | LeetCode 743 Network Delay • 1631 Path Min Effort • 787 Cheapest Flights • 1514 Max Probability |
|---|---|

## 5.17 Interval — Merge, Insert, Count Rooms

```
# Merge overlapping intervals
def merge(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
```

```
    for s, e in intervals[1:]:
        if s <= merged[-1][1]: merged[-1][1] = max(merged[-1][1], e)
        else:                  merged.append([s, e])
    return merged

# Minimum meeting rooms needed
import heapq
def min_rooms(intervals):
    intervals.sort(key=lambda x: x[0])
    heap = []
    for s, e in intervals:
        if heap and heap[0] <= s: heapq.heapreplace(heap, e)
        else:                     heapq.heappush(heap, e)
    return len(heap)

# Remove minimum intervals to eliminate all overlaps
def erase_overlap(intervals):
    intervals.sort(key=lambda x: x[1])  # sort by end
    count, last_end = 0, float('-inf')
    for s, e in intervals:
        if s >= last_end: last_end = e
        else:             count += 1
    return count
```

| ✓Solves | LeetCode 56 Merge • 57 Insert • 253 Meeting Rooms II • 435 Non-overlapping • 452 Min Arrows |
|---|---|

## 5.18 Matrix DFS / BFS

```
dirs = [(0,1),(0,-1),(1,0),(-1,0)]  # 4-directional

# DFS flood fill
def dfs_grid(grid, r, c, visited):
    rows, cols = len(grid), len(grid[0])
    if r<0 or r>=rows or c<0 or c>=cols: return
    if (r,c) in visited or grid[r][c]==BLOCKED: return
    visited.add((r,c))
    for dr,dc in dirs:
        dfs_grid(grid, r+dr, c+dc, visited)

# BFS shortest path
from collections import deque
def bfs grid(grid, start, end):
    rows, cols = len(grid), len(grid[0])
    visited = set([start])
    queue   = deque([(start[0], start[1], 0)])
    while queue:
        r,c,dist = queue.popleft()
        if (r,c)==end: return dist
        for dr,dc in dirs:
            nr,nc = r+dr, c+dc
            if (0<=nr<rows and 0<=nc<cols
                and grid[nr][nc]!=BLOCKED
                and (nr,nc) not in visited):
```

```
                visited.add((nr,nc))
                queue.append((nr,nc,dist+1))
    return -1
```

| | |
|---|---|
| ✓Solves | LeetCode 200 Number of Islands • 994 Rotting Oranges • 1091 Shortest Path Binary Matrix • 695 Max Area |

---

**End of Section 5.** Continue with Section 6 — Time Complexity Cheat Sheet.

# Python & Algorithms for Coding Interviews

Section 6 — Time Complexity Cheat Sheet

## 6.1 Data Structure Operations

| Data Structure | Operation | Average | Worst | Space |
|---|---|---|---|---|
| Array / List | Access by index | O(1) | O(1) | O(n) |
| | Append (end) | O(1) amort | O(n) | |
| | Insert / Delete (mid) | O(n) | O(n) | |
| | Search (unsorted) | O(n) | O(n) | |
| Hash Map / Set | Get / Set / Delete | O(1) | O(n) | O(n) |
| | Membership (in) | O(1) | O(n) | |
| Stack (list) | Push / Pop / Peek | O(1) | O(1) | O(n) |
| Queue (deque) | Enqueue / Dequeue | O(1) | O(1) | O(n) |
| Heap (heapq) | Push / Pop | O(log n) | O(log n) | O(n) |
| | Peek min / Heapify | O(1) / O(n) | | |
| Linked List | Insert at head | O(1) | O(1) | O(n) |
| | Search / Access | O(n) | O(n) | |
| BST (balanced) | Search / Insert | O(log n) | O(n)* | O(n) |
| | * worst = skewed tree | | | |
| Trie | Insert / Search | O(L) | O(L) | O(N·L) |
| Graph (adj list) | DFS / BFS | O(V+E) | O(V+E) | O(V+E) |

## 6.2 Sorting Algorithms

| Algorithm | Best | Average | Worst | Space | Stable? | Notes |
|---|---|---|---|---|---|---|
| Timsort (Python default) | O(n) | O(n log n) | O(n log n) | O(n) | Yes | list.sort(), sorted() |
| Merge Sort | O(n log n) | O(n log n) | O(n log n) | O(n) | Yes | Divide & conquer |
| Quick Sort | O(n log n) | O(n log n) | $O(n^2)$ | O(log n) | No | In-place, pivot |
| Heap Sort | O(n log n) | O(n log n) | O(n log n) | O(1) | No | Uses heap |
| Insertion Sort | O(n) | $O(n^2)$ | $O(n^2)$ | O(1) | Yes | Good for small n or nearly sorted |
| Counting Sort | O(n+k) | O(n+k) | O(n+k) | O(k) | Yes | Integer keys only |

| | |
|---|---|
| **Python Tip** | Python's list.sort() and sorted() use Timsort — O(n log n) worst, O(n) best on nearly sorted data. Always use the built-in unless the problem asks you to implement sorting yourself. |

## 6.3 Algorithm Pattern Complexity

| Pattern | Time | Space | Key Note |
|---|---|---|---|
| Two Pointers | O(n) or O(n log n)* | O(1) | * includes sort if needed |
| Sliding Window | O(n) | O(k) | k = window or alphabet size |
| Prefix Sum | O(n) build, O(1) query | O(n) | |
| Hash Map Counting | O(n) | O(n) | |
| Binary Search | O(log n) | O(1) | |
| Binary Search on Answer | O(n log R) | O(1) | R = answer range |
| Monotonic Stack | O(n) | O(n) | Each element pushed and popped once |

| | | | |
|---|---|---|---|
| Monotonic Queue | O(n) | O(k) | k = window size |
| Fast & Slow Pointers | O(n) | O(1) | |
| DFS (tree) | O(n) | O(h) | h = tree height |
| DFS (graph) | O(V+E) | O(V) | |
| BFS | O(V+E) | O(V) | |
| Backtracking (subsets) | O(2^n) | O(n) | |
| Backtracking (perms) | O(n!) | O(n) | |
| DP 1D | O(n) | O(n) | O(1) with rolling variable |
| DP 2D | O(m·n) | O(m·n) | O(n) with rolling row |
| Greedy (with sort) | O(n log n) | O(1) | |
| Union Find | O(α(n)) | O(n) | α = inverse Ackermann ≈ O(1) |
| Topological Sort | O(V+E) | O(V+E) | |
| Dijkstra | O((V+E)logV) | O(V+E) | Min-heap required |
| Trie insert/search | O(L) | O(N·L) | L=word length, N=word count |

## 6.4 Python Built-in Operations

| Operation | Complexity | Note |
|---|---|---|
| len(x) | O(1) | Lists, dicts, sets, strings all cache length |
| list[i] | O(1) | Random access |
| list.append(x) | O(1) amort | Dynamic array |
| list.pop() | O(1) | From end |
| list.pop(0) | O(n) | □ Use deque.popleft() instead |
| list.insert(i, x) | O(n) | Shifts right of i |
| list.sort() | O(n log n) | Timsort, in-place |
| sorted(iterable) | O(n log n) | Returns new list |
| x in list | O(n) | Linear scan |
| x in set / dict | O(1) | Hash lookup |
| dict[k] = v / dict.get(k) | O(1) | |
| dict.items() iteration | O(n) | |

| | | |
|---|---|---|
| set.add() / set.discard() | O(1) | |
| set union / intersection | O(m+n) | m, n = sizes of sets |
| heapq.heappush | O(log n) | |
| heapq.heappop | O(log n) | |
| heapq.heapify | O(n) | Better than n pushes |
| bisect.bisect_left | O(log n) | Binary search on sorted list |
| bisect.insort | O(n) | O(log n) search + O(n) insert shift |
| deque.appendleft / popleft | O(1) | list.pop(0) is O(n)! |
| Counter(iterable) | O(n) | |
| ".join(list) | O(n) | Single allocation — always use this |
| str += str (in loop) | O($n^2$) total | □ Creates new string every time |

## 6.5 Complexity Decision Guide

| n (input size) | Max complexity you can afford | Patterns that work |
|---|---|---|
| n ≤ 10 | O(n!) | Any — brute force, all permutations |
| n ≤ 20 | O(2^n) | Backtracking, bitmask DP |
| n ≤ 100 | O($n^3$) | 3-nested loops, Floyd-Warshall, O($n^3$) DP |
| n ≤ 1,000 | O($n^2$) | Nested loops, O($n^2$) DP, insertion sort |
| n ≤ 100,000 | O(n log n) | Sorting, Binary Search, Heap, Divide & Conquer |
| n ≤ 1,000,000 | O(n) | Two Pointers, Sliding Window, Hash Map, Prefix Sum, Linear DP |
| n > 10,000,000 | O(n) or O(log n) | Binary Search, Counting Sort, single-pass algorithms |

## 6.6 Big-O Growth Rate Reference

| Complexity | Name | n=10 | n=100 | n=1,000 | n=1,000,000 |
|---|---|---|---|---|---|
| O(1) | Constant | 1 | 1 | 1 | 1 |
| O(log n) | Logarithmic | 3 | 7 | 10 | 20 |
| O(n) | Linear | 10 | 100 | 1,000 | 1,000,000 |
| O(n log n) | Linearithmic | 33 | 664 | 10,000 | 20,000,000 |
| $O(n^2)$ | Quadratic | 100 | 10,000 | 1,000,000 | $10^{12}$ ✕ |
| $O(n^3)$ | Cubic | 1,000 | 1,000,000 | $10^9$ ✕ | $10^{18}$ ✕ |
| O(2^n) | Exponential | 1,024 | $10^{30}$ ✕ | — | — |
| O(n!) | Factorial | 3.6M | — | — | — |

| | |
|---|---|
| **Interview Rule** | If your solution is $O(n^2)$ or worse and n > 10,000 — you will almost certainly get TLE (Time Limit Exceeded). Always check constraints first and target the correct complexity before writing a single line of code. |

## 6.7 Master Quick Reference

| Problem type | Target complexity | Pattern / Tool |
|---|---|---|
| Pair in sorted array | O(n) | Two Pointers |
| Subarray sum / count subarrays | O(n) | Prefix Sum + Hash Map |
| Longest / shortest substring | O(n) | Sliding Window |
| Frequency / grouping / lookup | O(n) | Hash Map / Counter |
| Search in sorted data | O(log n) | Binary Search |
| Next greater / span | O(n) | Monotonic Stack |
| Sliding window max/min | O(n) | Monotonic Deque |
| Cycle or middle in linked list | O(n) | Fast & Slow Pointers |
| Connected regions on grid/graph | O(V+E) | DFS |
| Shortest path (unweighted) | O(V+E) | BFS |
| All combinations / subsets | O(2^n) | Backtracking |
| Minimum cost / count ways / optimal | O(n·states) | DP |
| Interval merge / scheduling | O(n log n) | Greedy + Sort |
| Group / union elements dynamically | O(α(n)) | Union Find |

| | | |
|---|---|---|
| Task ordering / prerequisites | O(V+E) | Topological Sort |
| Shortest path (weighted graph) | O((V+E)logV) | Dijkstra |
| Prefix word match / autocomplete | O(L) | Trie |
| Top K elements / K-th largest | O(n log k) | Heap |

---

**Book Complete.** You now have everything needed to recognise patterns, apply templates, and solve most LeetCode interview problems efficiently. Revise Section 4 before every session. Revise Section 6 before every interview. Practice daily.